

# PALIMPSESTE

CAHIER DE RÉALISATION / SP-1.0

# Du dessin au sort.

La boucle complète de création  
et d'exécution des sorts sous Unity.

---

19 SEPTEMBRE 2026 · UNITY · WINDOWS x64

Image réelle → interprétation multimodale → description → plan de sort → compilation contrôlée  
→ exécution Unity.

25 chapitres · 4 annexes · 30 tickets de construction  
Architecture, contrats, prompts, physique, présentation et recette.

Une version finale du sous-système, pas le jeu entier.

Ce dossier définit le logiciel à réaliser. Il ne constitue pas un build Unity ou une validation de la qualité d'un modèle. Les décisions issues de vos échanges et les nouveaux arbitrages sont distingués.

## TABLE DES MATIÈRES

# Construire la boucle.

|    |                                                       |    |
|----|-------------------------------------------------------|----|
| 01 | Le produit à livrer                                   | 3  |
| 02 | Ce qui vient de vous, et ce que ce document arbitre   | 4  |
| 03 | L'architecture, de bout en bout                       | 6  |
| 04 | Stack et organisation du dépôt                        | 8  |
| 05 | Parcours utilisateur et écrans                        | 10 |
| 06 | Dessin, encre et irréversibilité                      | 12 |
| 07 | Moteur A : image vers description                     | 14 |
| 08 | Moteur B : description vers recette de sort           | 16 |
| 09 | Catalogue de capacités à livrer entièrement           | 18 |
| 10 | Du dessin à la géométrie 3D                           | 20 |
| 11 | Effets, physique et réactions                         | 22 |
| 12 | Compilation, bornes et paquet immuable                | 24 |
| 13 | Exécution Unity : classes, événements et cycle de vie | 27 |
| 14 | Rendu, son et qualité de présentation                 | 29 |
| 15 | Scénarios de référence et exemple complet             | 30 |
| 16 | API du service de génération                          | 32 |
| 17 | Persistence, reprise et unicité                       | 34 |
| 18 | Sécurité, secrets et coût                             | 36 |
| 19 | Installation, exploitation et récupération            | 38 |
| 20 | Performance et observabilité                          | 40 |
| 21 | Plan de tests et validation humaine                   | 41 |
| 22 | Ordre de réalisation et jalons                        | 43 |
| 23 | Définition de terminé et intégration au futur jeu     | 45 |
| 24 | Consigne à donner à l'agent de développement          | 46 |
| 25 | Sources et traçabilité                                | 47 |
| A  | Règles des contrats                                   | 49 |
| B  | Branchement réel du fournisseur                       | 54 |
| C  | Les 30 tickets de construction                        | 57 |
| D  | Consigne prête pour le développeur                    | 60 |

**CHAPITRE 01 / SP-1.0**

# Le produit à livrer

## Une fonctionnalité terminée, dans un périmètre volontairement réduit

Le livrable logiciel attendu est une application Windows x64 construite avec Unity, accompagnée de son service de génération. Elle comporte un atelier de dessin, une bibliothèque de parchemins, une fiche de sort et une scène 3D d'épreuve. Cette scène sert à observer les propriétés du sort : ce n'est pas une proposition de monde, de campagne ou de système de combat définitif.

La boucle est terminée seulement lorsqu'un dessin inédit produit, avec un vrai appel multimodal, un sort inédit composé de capacités prises en charge, visible et utilisable dans un build autonome. Le résultat doit avoir une géométrie, des collisions, des règles de ciblage, des effets mesurables, une durée de vie, un rendu et un son. Il doit survivre à la fermeture du logiciel et rester lançable sans nouvel appel au modèle.

**Ce qui ne suffit pas :** une réponse JSON affichée dans une interface ; une boule lumineuse quel que soit le dessin ; un switch entre quelques sorts complets préfabriqués ; des coordonnées de traits utilisées à la place d'une lecture de l'image ; un mode « simulation » présenté comme une génération réelle ; un résultat qui ne fonctionne que dans l'éditeur Unity.

## Dans SP-1.0

L'application doit fournir la saisie souris et stylet, trois styles de traces, quatre encres, un layout de trois régions, l'engagement irréversible du parchemin, deux étapes de génération séparables, un compilateur contrôlé, six porteurs d'effets composables, les effets et réactions du chapitre 11, une scène de mesure, la sauvegarde, la reprise des tâches techniques, l'export d'un dossier de revue, les tests et l'installation documentée.

## En dehors de SP-1.0

Pas de monde ouvert, de progression de personnage, de rareté économique, de commerce, de classes, de slots de combat définitifs, de matchmaking ou de PvP. Pas de génération de C#, de shaders, de DLL, de créatures animées ou de nouvelles lois physiques à l'exécution. Pas de simulation générale de fluides ou de destruction du décor. Pas de WebGL ou de mobile à livrer. Ces exclusions n'autorisent pas à remplacer la boucle par une maquette.

Le laboratoire donne accès à des parchemins neufs pour les essais. **Ce stock de test n'est pas un brouillon gratuit du futur jeu :** chaque parchemin engagé demeure irréversible et chaque nouvelle création possède un nouvel identifiant.

## CHAPITRE 02 / SP-1.0

# Ce qui vient de vous, et ce que ce document arbitre

## Priorité des sources

Votre demande actuelle choisit Unity et étend le périmètre jusqu'au sort lançable. Elle remplace donc la limite précédente « jusqu'à la description ». Le transcript de révision du 19 septembre prévaut sur le premier brainstorm et sur les propositions rejetées de la V1 du book. Les extraits cités ci-dessous constituent le socle ; les règles chiffrées et détails d'implémentation sont de nouvelles propositions de réalisation, pas des décisions que vous auriez déjà prises.

| ID  | Décision acquise                                                                                                        | Origine                                                |
|-----|-------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| U01 | Unity ; boucle complète dessin → description → sort réel, sans construire le reste du jeu.                              | Demande actuelle.                                      |
| U02 | Le modèle interprète l'image directement, avec une image de référence et une image du dessin.                           | Révision, 05:05:24-05:06:13. [T2]                      |
| U03 | Les propriétés et mécaniques implémentées se composent ; pas de table de sorts complets et pas de code libre généré.    | Révision, 04:31:47-04:34:46. [T2]                      |
| U04 | Pas de brouillon, de gomme, d'annulation, de presets de sorts ou de confirmation après découverte du résultat.          | Révision, 04:53:34-04:56:35. [T2]                      |
| U05 | Une trace ratée peut produire un mauvais sort. Le pinceau compte par sa marque visible, pas par une intention garantie. | Révision, 04:51:59-04:52:14 et 04:55:45-04:56:35. [T2] |
| U06 | Fermer un parchemin encre conserve l'engagement et le contenu inscrit.                                                  | Révision, 04:58:04-04:59:34. [T2]                      |
| U07 | Reproduire un dessin doit pouvoir donner un résultat proche, pas une recette de copie exacte.                           | Révision, 04:35:11-04:37:54. [T2]                      |
| U08 | Les créateurs humains valident la qualité des tests ; l'agent ne s'auto-attribue pas cette validation.                  | Révision, 04:42:07-04:42:29. [T2]                      |
| U09 | Les sorts doivent pouvoir agir sur la physique du monde.                                                                | Révision, 05:02:32-05:03:42. [T2]                      |

## Arbitrages proposés et utilisés dans ce cahier des charges

Les choix suivants donnent une base directement implémentable. Ils restent identifiés comme propositions afin qu'un changement soit conscient et tracé.

| ID  | Choix de réalisation SP-1.0                                                                                              | Motif et conséquence                                                                                                   |
|-----|--------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| P01 | Unity 6.3 LTS, URP, Windows x64 ; serveur ASP.NET Core / .NET 10 ; noyau partagé en C# compatible .NET Standard 2.1.     | Un langage pour les contrats et les règles. Le runtime Unity n'est pas le runtime .NET 10. [W1-W4]                     |
| P02 | Un geste continu par région. Au relâchement, toutes les régions effectivement encrées par ce geste se verrouillent.      | Interprétation opérationnelle des formulations du transcript sur le lever du pinceau. Aucune gomme cachée.             |
| P03 | Deux étapes de modèle : lecture multimodale, puis traduction textuelle contrôlée ; compilation déterministe ensuite.     | Permet d'identifier si une erreur vient de la lecture ou de sa traduction en mécanique.                                |
| P04 | Trois régions : noyau, couronne, périphérie ; elles orientent respectivement matière/effet, déploiement et comportement. | Une grammaire d'apprentissage proposée, non une classification automatique par coordonnées.                            |
| P05 | Variations propres au parchemin, figées à la création, surtout dans la signature visuelle.                               | Proximité conservée ; aucune relance au moment du lancement. Pas de promesse mathématique d'unicité mécanique infinie. |
| P06 | Le temps d'attente du fournisseur ne détermine pas le cooldown.                                                          | Une panne réseau ne doit pas modifier les règles d'un sort. La piste du transcript était explicitement ouverte.        |
| P07 | Sort déjà créé lançable hors ligne ; nouvelle interprétation nécessitant le service.                                     | L'IA n'est pas dans la boucle de simulation du combat.                                                                 |
| P08 | Scène d'épreuve 3D avec caméra orbitale et lanceur fixe repositionnable.                                                 | Observer les sorts sans choisir aujourd'hui la caméra et les déplacements du futur jeu.                                |
| P09 | Catalogue de capacités fermé pour cette livraison, composition ouverte dans ce catalogue.                                | Toute capacité annoncée doit avoir son code, ses assets, ses limites et ses tests.                                     |
| P10 | Une panne technique reste une panne technique ; aucun « mauvais sort » artificiel pour la cacher.                        | L'échec artistique reste possible ; le système ne rembourse pas le dessin et ne falsifie pas un succès.                |

## CHAPITRE 03 / SP-1.0

# L'architecture, de bout en bout

## Trois transformations et une exécution

```

UNITY : DESSIN + JOURNAL DURABLE
  ↓ capture figée / image de référence versionnée
SERVICE A : DrawingInterpreter (multimodal)
  ↓ SpellDescription : texte + clauses + faits + liens au dessin
GEOMETRY RESOLVER : extraction de formes dans le dessin réel
  ↓ banque de géométries mesurées, sans reconnaissance de sorts
SERVICE B : DescriptionPlanner (texte + catalogue + géométries)
  ↓ SpellPlan : porteurs, événements, effets, paramètres
COMPILATEUR C# : validation, quantification, bornes, traçabilité
  ↓ CompiledSpell + texte factuel dérivé des règles finales
UNITY : SpellRuntime → collisions / effets / rendu / audio

```

**A interprète.** Il reçoit l'image entière, pas seulement une liste de points ou quatre recadrages. Il peut lire le noyau différemment de la périphérie tout en conservant la continuité du dessin. Il choisit une interprétation, pas un menu de candidats soumis au joueur. Les observations qu'il donne sont des explications visibles et vérifiables, pas une demande de raisonnement interne.

**B traduit.** Il transforme la description figée en une recette exécutable dans le catalogue réellement présent. Il ne reçoit pas de nouvelle image pour réinventer le sort. Il peut choisir comment réaliser un comportement, mais ne doit pas inventer une nouvelle intention, ajouter un soin ou retirer un déclenchement. Ses paramètres sont des données, jamais des instructions C#.

**Le compilateur contrôle.** Il refuse une recette technique incorrecte, vérifie les références, calcule les bornes de coût d'exécution, normalise les unités et produit un paquet immuable. La validation d'un schéma n'est pas une validation de la fidélité artistique ; les deux doivent rester séparées.

**Unity exécute.** Il consomme le paquet, instancie des porteurs génériques, évalue les collisions et applique les effets. Il ne contacte pas le modèle au moment d'un lancement. Un même paquet rejoué dans une scène identique conserve ses paramètres ; on ne promet pas pour autant un déterminisme physique bit à bit entre toutes les machines.

## La description n'est pas une simple phrase publicitaire

`SpellDescription` contient un texte lisible et des clauses identifiées. Chaque clause comporte des faits contrôlables : type de porteur, famille d'effet, cible, événement, mouvement ou région géométrique. Cette enveloppe évite qu'une phrase telle que « un voile protecteur qui repousse » devienne arbitrairement un bouclier absorbant et un soin.

Une **description rédigée par un concepteur** est également testable indépendamment de A : le concepteur renseigne texte, clauses, faits et relations dans le même contrat, puis l'envoie à B par `POST /v1/authoring/plan`. Il peut partir d'une description existante, dans un dossier de diagnostic distinct. Ce chemin reste marqué `text_authoring` et n'est pas accessible depuis un parchemin engagé. La conversion d'une simple phrase libre sans ces champs en contrat serait une normalisation supplémentaire ; elle n'est pas nécessaire à SP-1.0 et ne doit pas être annoncée comme livrée. Le moteur B reçoit bien une description, et non des instructions de code.

## Un texte fiable à l'arrivée

Le texte de A est l'interprétation enregistrée. La fiche de règles affichée au joueur est produite par **R** **u**les**P**resenter à partir du paquet compilé : nombre de projectiles, portée, ciblage, déclenchement, durée, dégâts, statut, exclusions. Un titre évocateur peut venir de A, mais la prose non contrôlée ne peut pas promettre un effet absent. Une divergence entre les clauses et la recette interdit la publication jusqu'à réparation technique ou examen du cas.

## CHAPITRE 04 / SP-1.0

# Stack et organisation du dépôt

## Versions

Unity 6.3 LTS est la ligne recommandée, pas un numéro de correctif supposé être le plus récent. Le développeur installe un correctif 6000.3 disponible et testé, puis enregistre sa version exacte dans `ProjectVersion.txt`. URP, Input System et les autres packages sont résolus dans cet éditeur et verrouillés par `manifest.json` et `packages-lock.json`. Aucun changement automatique de moteur au milieu du projet. [W1, W2, W5]

Le serveur cible .NET 10. Les bibliothèques partagées ciblent **.NET Standard 2.1**, emploient une syntaxe C# compatible avec l'éditeur retenu et ne référencent ni ASP.NET, ni Entity Framework, ni `UnityEngine`. La documentation Unity distingue explicitement ses profils d'API des bibliothèques compilées pour .NET Core. [W3, W4]

Le rendu s'appuie sur URP, des meshes procéduraux, des matériaux et systèmes de particules contrôlés. VFX Graph peut enrichir le rendu après validation de sa compatibilité, mais aucune mécanique ne doit en dépendre. Pas d'ECS obligatoire : commencer avec un gestionnaire central et des objets réutilisés, puis mesurer avant de changer d'architecture.

## Arborescence à réaliser

|                                               |                                                  |
|-----------------------------------------------|--------------------------------------------------|
| /game/                                        | projet Unity                                     |
| Assets/Palimpseste/                           |                                                  |
| Bootstrap/ Drawing/ Library/ SpellRuntime/    |                                                  |
| Physics/ Presentation/ Review/ Editor/ Tests/ |                                                  |
| Scenes/Bootstrap.unity                        |                                                  |
| Scenes/SpellLab.unity                         |                                                  |
| Packages/                                     | dépendances verrouillées                         |
| /shared/Palimpseste.Contracts/                | DTO sans dépendance Unity                        |
| /shared/Palimpseste.Core/                     | raster, géométrie, compilation, règles           |
| /backend/Api/                                 | HTTPS, autorisations, uploads, lecture           |
| /backend/Worker/                              | interprétation, planification, reprise           |
| /backend/Infrastructure/                      | PostgreSQL, stockage, fournisseur                |
| /catalog/                                     | capacités, matériaux, règles de laboratoire      |
| /prompts/                                     | versions de A, B, réparation                     |
| /contracts/                                   | JSON Schema et OpenAPI                           |
| /tests/                                       | contrats, fixtures, intégration, corpus          |
| /ops/                                         | composition de services, sauvegarde, déploiement |
| /docs/                                        | décisions, démarrage, exploitation, remise       |

Les dossiers `/shared` sont importés dans Unity comme packages locaux ou assemblées construites de manière reproductible. Choisir une seule méthode et la verrouiller. Les DTO JSON ont des noms et unités identiques des deux côtés. Pour Unity, utiliser un sérialiseur JSON explicitement compatible avec le build IL2CPP retenu ; déclarer les types et préserver les membres nécessaires. Une réussite sous Mono dans l'éditeur n'est pas une preuve de réussite IL2CPP. [W6]

## Contrats de modules

| Module             | Entrée → sortie                                          | Interdiction importante                                       |
|--------------------|----------------------------------------------------------|---------------------------------------------------------------|
| DrawingSession     | événements pointeur → journal, masque et états           | Ne décide pas des dégâts ni du rôle du sort.                  |
| CaptureService     | journal figé → PNG canonique + empreintes                | Ne modifie pas le dessin pour le rendre « meilleur ».         |
| DrawingInterpreter | deux images + prompt → description                       | Ne choisit pas un prefab de sort complet.                     |
| GeometryResolver   | pixels + indices régionaux → géométries                  | Ne fait pas de classification cercle = bouclier.              |
| DescriptionPlanner | description + capacités → plan                           | Ne change pas la description figée pour justifier sa recette. |
| SpellCompiler      | plan + géométries + profils → paquet                     | N'exécute jamais le texte de modèle.                          |
| SpellRuntime       | paquet + contexte de lancement → événements              | Ne contient aucune clé de fournisseur.                        |
| ReviewRecorder     | observations humaines → dossier signé par un utilisateur | L'agent ne renseigne pas « approuvé » à la place de l'équipe. |

## CHAPITRE 05 / SP-1.0

# Parcours utilisateur et écrans

## Écran 1 — Bibliothèque

Liste des parchemins : vierge, encre localement, transmission en attente, interprétation, construction, disponible ou incident technique. Chaque carte montre la vignette réelle et son état textuel. Le tri, l'ouverture d'une fiche et le lancement d'un sort disponible fonctionnent hors ligne. Une déconnexion ne masque pas une tâche déjà engagée.

L'action « Nouveau parchemin » alloue un support de laboratoire. Elle ne duplique pas le contenu d'un support existant. Aucun bouton « Régénérer », « Essayer une autre version » ou « Corriger mon dessin » sur les parchemins encrés.

## Écran 2 — Dessin

Le parchemin occupe la zone centrale ; les trois régions sont identifiables sans submerger le dessin. À côté : styles de traces, palette d'encres, épaisseur, quantité d'encre restante et une phrase permanente : « **Une région encrée se verrouille lorsque vous relevez le pinceau.** » Ce texte doit correspondre exactement à P02.

Les contrôles modifient uniquement la trace à venir. Ils ne choisissent ni dégâts, ni soin, ni bouclier. Le pinceau double produit physiquement deux lignes. Un gribouillage qui les fusionne sera transmis tel quel au modèle, même si le joueur avait choisi ce pinceau.

Aucun aperçu des effets du sort n'apparaît avant l'engagement final du dessin. Un aperçu du diamètre du pinceau sans peinture est autorisé. Le zoom et le déplacement du papier n'altèrent pas les coordonnées du dessin.

## Écran 3 — Interprétation et fiche

Pendant la génération, afficher la vraie étape et un temps écoulé, pas un faux pourcentage. À réception du paquet complet : titre, dessin, description, propriétés vérifiées et action « Lancer dans le laboratoire ». La sortie partielle de A peut être consultée en mode concepteur avec la mention « Interprétation intermédiaire — règles non encore disponibles ». Elle n'est pas une confirmation à valider par le joueur.

## Écran 4 — Scène d'épreuve

Le lanceur est un socle ou avatar neutre ; souris pour viser, clic pour lancer, touche Échap pour quitter le mode de visée. Caméra orbitale, remise à zéro du terrain et boutons de scénarios sont des outils d'observation. Leur utilisation ne relance pas le modèle et ne modifie pas le parchemin.

Le panneau latéral affiche les mesures : cibles touchées, dégâts/soins effectifs, vitesse des objets poussés, états appliqués, nombre d'instances et coût de simulation. Un mode masque ces mesures pour juger le ressenti visuel. L'accès au mode concepteur est explicite ; il ne doit pas devenir une interface de jeu surchargée.

## États et accessibilité

Prévoir souris absente, perte de focus, stylet retiré, écran redimensionné, changement de résolution et contrôle clavier de tous les éléments hors dessin. Les régions verrouillées sont signalées par un contour et un libellé, pas seulement une couleur. Les couleurs des encres possèdent un nom lisible. L'interface est livrée en français ; les identifiants techniques restent en anglais.

## CHAPITRE 06 / SP-1.0

# Dessin, encre et irréversibilité

## Layout proposé `three_regions_v1`

Le canvas canonique mesure  $1\,024 \times 1\,024$  pixels. Les positions sont conservées en coordonnées normalisées entières de 0 à 65 535. Pour la définition des masques, `u` et `v` vont de 0 à 1, avec origine en haut à gauche ;  $\rho = \sqrt{(2u-1)^2 + (2v-1)^2}$ . Le noyau correspond à  $\rho < 0,32$ , la couronne à  $0,32 \leq \rho < 0,68$  et la périphérie à  $0,68 \leq \rho \leq 1$ . L'extérieur du disque n'accepte pas d'encre.

Ces dimensions sont P04, pas une géographie obligatoire de tous les futurs parchemins. La définition est versionnée. La même définition fabrique les masques, l'image de référence, l'affichage et le texte de prompt. Les guides graphiques ne font jamais partie du masque d'encre exporté pour la géométrie.

## Sémantique précise d'un geste

Un geste commence à `PointerDown` dans une région ouverte et se termine à `PointerUp` ou lors d'une interruption définie. Il peut traverser plusieurs régions ouvertes. Les pixels qui débordent dans une région déjà verrouillée sont découpés ; aucun remplacement ni mélange n'y est autorisé. À la fin du geste, toutes les régions ayant réellement reçu de l'encre se verrouillent. Un pinceau double ou pointillé peut produire plusieurs fragments au cours du même geste : cela ne compte pas comme plusieurs gestes.

Un clic sans déplacement produit une empreinte. Un clic en dehors du disque ou intégralement sur une région verrouillée ne consomme pas d'encre et ne verrouille rien. Si un seul pixel d'encre admissible est déposé, le support est engagé. Un geste qui traverse les trois régions et se termine clôt le dessin. Sinon, le joueur peut continuer uniquement dans les régions encore vierges.

La fin intervient aussi à la fermeture de l'atelier après inscription, à une interruption de session imposant la clôture, ou lorsque l'encre est épuisée. L'épuisement est une règle de réalisation ajoutée ici : il clôt les régions restantes sans exiger un dernier clic impossible. Fermer un support resté parfaitement vierge ne le consomme pas.

Une perte de focus interrompt le geste comme un relâchement, mais ne ferme pas automatiquement tout l'atelier. Un crash ou une fermeture d'application clôt le support déjà encré à partir du dernier journal durable lors de la reprise ; il n'ouvre jamais une session de correction.

## Styles de traces et budget d'encre

Les trois styles à livrer sont `solid`, `double` et `dotted`. Ils partagent le même mécanisme d'échantillonnage spatial. L'épaisseur est continue dans une plage bornée ; la pression du stylet agit sur cette épaisseur. À la souris, une pression constante est utilisée. Le pointillé est défini par la distance parcourue, jamais par le nombre de frames, afin que 30 et 144 images/s n'en changent pas le motif.

Le budget du laboratoire est fixé à **1 000 unités par support**. Il est commun aux quatre couleurs. Une unité d'encre n'est ni un point de mana ni un point de dégâts. La progression du budget dans le futur jeu est hors périmètre.

Le coût est l'intégrale discrète de la surface déposée par les empreintes de pinceau, pondérée par leur opacité et leur coefficient d'encre, sur les seules régions ouvertes. Les passages superposés coûtent à nouveau, même si l'image est déjà saturée. Ne pas facturer seulement la surface unique occupée : cela permettrait de retracer gratuitement. Ne pas arrondir un coût minimal à chaque événement pointeur : la consommation dépendrait du matériel.

**Implémentation** : rééchantillonnage à pas spatial fixe lié au diamètre, accumulateur entier en micro-unités, arrondi de l'affichage seulement. La dernière empreinte est appliquée partiellement lorsque le budget est insuffisant, puis l'état est clôturé. L'algorithme et ses paramètres sont dans `BrushRasterizerVersion`; tests de même trajectoire à plusieurs fréquences d'entrée.

## Source de vérité visuelle

Le journal contient les points, le style physique du pinceau, la couleur visible, le diamètre, la pression et l'ordre des gestes. Il permet de reconstruire les pixels et de diagnostiquer un bug ; **il ne remplace jamais l'image à l'entrée du modèle.**

Le rasteriseur de référence produit un masque RGBA à partir des empreintes en arithmétique entière. Le rendu interactif peut utiliser une texture et des mises à jour par tuiles. Une accélération GPU est autorisée uniquement après comparaison au rasteriseur de référence ; exporter uniquement un framebuffer éclairé, compressé ou color-corrigé par la scène est interdit. Une lecture GPU asynchrone est un outil possible de capture, avec gestion de son erreur et de sa latence, pas une raison de perdre le dessin. [W7]

À la clôture, produire `drawing.png` sur fond neutre avec les mêmes guides que `reference.png`, `ink.png` sans guides, `capture.json`, le journal et les empreintes SHA-256. Conserver les couleurs sRGB, l'orientation et la résolution ; supprimer les métadonnées inutiles. Le hash de pixels RGBA décodés est distinct du hash des octets du fichier PNG : deux encodeurs peuvent produire des PNG différents pour les mêmes pixels.

## Journal durable et reprise

Écrire les points par petits blocs append-only avec compteur de séquence et checksum. Le premier bloc d'encre marque le support engagé. Conserver un checkpoint atomique et le journal restant. La fermeture normale attend la persistance locale, pas la réponse du fournisseur. Une file locale durable transmet ensuite le contenu et réutilise les mêmes identifiants.

Proposition de tolérance technique : checkpoint au plus toutes les 100 ms et à chaque fin de geste ; un arrêt brutal peut perdre les derniers points non durables, **jamais rendre le support vierge**. La reprise indique qu'un dessin partiel a été conservé. Ne pas prétendre garantir tous les derniers pixels face à une coupure électrique. Si le journal est irrécupérable, conserver un incident `capture_corrupted`, sans générer un faux résultat.

Ce laboratoire n'est pas un système anti-triche : effacer volontairement les fichiers locaux ou modifier un exécutable ne fait pas partie des garanties économiques de SP-1.0. Les transitions normales et les doublons réseau doivent en revanche être correctement protégés.

## CHAPITRE 07 / SP-1.0

# Moteur A : image vers description

## Entrées réellement envoyées

Le worker construit un appel contenant une instruction système versionnée, la définition textuelle du layout, le catalogue de capacités, **l'image 1 de référence et l'image 2 réellement dessinée**. Le modèle reçoit l'image entière. L'API de référence accepte plusieurs images dans une même requête ; ses sorties structurées permettent d'exiger un contrat JSON, sans supprimer le besoin de traiter refus et réponses incomplètes. [W8, W9]

L'image 1 est un guide spatial neutre, pas une image de boule de feu « cible ». L'image 2 emploie le même cadre, sans UI ni curseur. Les identifiants de pinceaux et leurs noms ne sont pas transmis comme rôle tactique. La matière d'une encre peut orienter une affinité, mais ne garantit jamais dégâts, soin ou protection.

## Responsabilités de A

A décrit ce qu'il voit, relie des traits à des propriétés et produit une seule interprétation cohérente. Le noyau oriente matière et effet ; la couronne oriente distribution et déploiement ; la périphérie oriente comportement et déclenchement. La lecture globale peut résoudre une ambiguïté, mais elle ne doit pas effacer systématiquement la grammaire proposée.

Il faut conserver les particularités visibles : courbure, rupture, densité, répétition, asymétrie, superposition, épaisseur et continuité. Le résultat peut être faible, court, discontinu ou peu utile. A ne demande pas de redessiner, n'accorde pas une utilité minimale garantie et ne transforme pas une image difficile en sort prédéfini de secours.

L'image peut porter du texte. Une instruction écrite dans le dessin est un contenu à interpréter, jamais une instruction prioritaire autorisant le modèle à sortir du catalogue, changer de schéma, révéler des secrets ou appeler des outils. Aucun outil de navigateur, shell ou exécution n'est fourni à A.

## Contrat `SpellDescription`

Le fichier `contracts/spell-description.schema.json` est la définition normative. Il comprend une version, un titre, un résumé, des observations et des clauses. Une observation désigne une région et une marque visible. Une clause contient une phrase de mécanique, ses observations d'origine et une liste de faits typés.

Un fait hérite du sujet logique de sa clause et possède une dimension et une valeur contrôlée : `carrier=field`, `effect=burn`, `target=hostile`, `motion=stationary`, `event=tick`, par exemple. Les valeurs numériques finales sont fixées dans B ; le texte de A indique des ordres de grandeur qualitatifs lorsqu'ils sont justifiés. Une clause `visual_only` peut expliciter une silhouette sans prétendre à une nouvelle mécanique.

Le tableau `relations` relie explicitement les sujets : source, événement, cible et nombre maximal d'activations. Il distingue la mécanique d'un porteur de la phrase « après son impact, créer un autre porteur ». Un fait `event` d'une clause concerne l'application d'un effet sur son sujet ; les événements de naissance d'un enfant se trouvent dans `relations`.

Le modèle renseigne aussi les correspondances entre régions et usages géométriques. A peut demander `ring/path` pour une trajectoire ou `outer/footprint` pour une zone. Il ne fabrique pas une liste de coordonnées supposées fidèles aux pixels : le résolveur du chapitre 10 réalise cette partie.

## Qualité et choix du modèle

Le fournisseur est derrière `IMultimodalInterpreter`. Un modèle multimodal généraliste suffisamment capable est utilisé en configuration initiale ; `gpt-6-astra` constitue un candidat documenté, pas un choix de qualité déjà validé sur vos dessins. Le modèle de B est configurable séparément. Les identifiants disponibles, leurs capacités et les paramètres acceptés doivent être testés avec la clé de l'équipe ; l'accès à un produit de discussion ne constitue pas une preuve d'accès à un modèle API donné. [W10]

Ne pas construire d'abord une usine de comparaison : un fournisseur fonctionne réellement, puis deux configurations maximum sont comparées sur le même corpus. Un changement de modèle ou de prompt crée une nouvelle version et exige une revue humaine. Ne pas modifier silencieusement les anciens parchemins.

## CHAPITRE 08 / SP-1.0

# Moteur B : description vers recette de sort

## Entrée figée, responsabilité limitée

B reçoit `SpellDescription`, la banque de géométries effectivement extraite, le catalogue des porteurs et effets, les paramètres du laboratoire et son schéma de sortie. Il ne reçoit ni code du projet, ni accès à Unity, ni identifiants de prefabs arbitraires, ni permission de modifier le catalogue.

B doit produire un **plan déclaratif**. Il choisit des porteurs génériques et les lie par événements. Exemple : un projectile courbe rencontre une cible ; cette rencontre fait apparaître une zone reprenant un contour du dessin ; la zone applique une brûlure à cadence définie. Aucun sort complet pré-écrit ne correspond à cet assemblage.

La fidélité de B se mesure par la conservation des clauses, pas par le style de sa prose. Chaque porteur et chaque effet citent les clauses qui les justifient. Une clause mécanique doit être satisfaite ou explicitement signalée comme non représentable. Une clause non représentable n'est jamais supprimée discrètement.

## Contrat `SpellPlan`

Le schéma livré décrit une liste de nœuds. Chaque nœud est un porteur d'effets : `projectile`, `beam`, `field`, `pulse`, `barrier` ou `trap`. Il dispose d'un déclencheur, d'un ancrage, d'une géométrie, d'options propres à son porteur, d'effets et de références aux clauses de A.

Un déclencheur dit : « au lancement » ou « lorsque tel événement se produit sur tel nœud parent ». Il définit son délai et le nombre maximal d'activations. Un enfant ne peut référencer qu'un parent existant ; le graphe doit être acyclique. L'ordre de la liste JSON n'est pas l'ordre d'exécution.

Les options sont typées par porteur. Un champ de projectile n'a aucune signification cachée sur une barrière. Les effets possèdent un événement, un filtre de cible, une quantité, une durée et, si nécessaire, une direction. Les valeurs inutiles pour un type doivent être nulles ou absentes selon son schéma, jamais ignorées silencieusement.

## Conservation mécanique et limites de la vérification

Le validateur compare les faits de A au plan : type de porteur, effets, filtres, événements et mouvements. Les familles d'effets d'un sujet doivent correspondre exactement ; pas de soin supplémentaire « parce que c'est plus intéressant ». Les événements et les références de géométrie doivent avoir une correspondance explicite. Les nombres restent dans les intervalles du profil autorisé.

Ces contrôles n'établissent pas une preuve générale d'équivalence entre langage naturel et simulation. Un texte peut être compris de travers malgré des champs corrects. Les humains doivent donc examiner la description, les liens au dessin et une vidéo du comportement. La fiche factuelle est, elle, générée depuis les règles finales pour éviter une divergence d'affichage.

## Réparation technique, pas reroll artistique

Si le JSON est invalide ou qu'une référence manque, la tentative est archivée et une réparation reçoit le même document de A, le plan fautif et une liste d'erreurs précise. Deux réparations au maximum par étape sont autorisées par défaut. Elles ne modifient pas le dessin, ne présentent aucun choix au joueur et ne recherchent pas un sort « plus puissant ».

Si un plan valide ne peut pas être produit, la tâche passe en incident récupérable. Le support reste engagé. Un opérateur peut reprendre le travail technique sur le même dessin, avec une trace de changement de version. Il ne doit pas inventer un succès pour satisfaire un indicateur de taux de génération.

## CHAPITRE 09 / SP-1.0

# Catalogue de capacités à livrer entièrement

## Portée de la promesse

SP-1.0 promet de réaliser les compositions admises par ce catalogue, pas n'importe quelle phrase imaginable. Un dragon dessiné peut donner une silhouette ou une trajectoire ; il ne devient pas une créature intelligente sans capacité d'invocation. Les exemples de tests ne sont pas proposés au joueur comme des recettes ou presets.

Le catalogue est une donnée versionnée dont chaque entrée possède : contrat, unités, limites, effets et événements compatibles, implémentation runtime, géométrie, rendu, audio et tests. Une capacité non terminée n'est pas annoncée au modèle dans un build candidat final. Réduire le catalogue contractuel nécessite une modification explicite de ce document, pas un commentaire TODO.

## Six porteurs, aucune liste de sorts complets

| Porteur    | Fonction réelle                                                                                    | Événements émis            | Options à réaliser                                                                    |
|------------|----------------------------------------------------------------------------------------------------|----------------------------|---------------------------------------------------------------------------------------|
| projectile | Volume mobile balayé à chaque tick ; trajectoire droite, courbe ou guidée.                         | spawn, hit, expire         | Rayon, vitesse, portée, durée, courbe, guidage, rebonds et pénétrations bornés.       |
| beam       | Faisceau instantané ou entretenu, bloqué par le premier obstacle ; relais optionnels entre cibles. | spawn, hit, tick, expire   | Portée, largeur, cadence, nombre de sauts, rayon de recherche et ligne de vue.        |
| field      | Empreinte au sol, potentiellement discontinue, avec hauteur d'influence.                           | spawn, enter, tick, expire | Forme issue de l'encre, durée, cadence, cible ; pas de mur solide implicite.          |
| pulse      | Front d'onde qui progresse dans une empreinte ou un disque ; une rencontre par cible.              | spawn, hit, expire         | Portée, durée d'expansion, largeur du front, forme, direction d'impulsion.            |
| barrier    | Obstacle physique temporaire avec points de structure, dessiné comme un ensemble de segments.      | spawn, block, expire       | Forme, hauteur, épaisseur, durée et capacité d'absorption ; pas de dégâts implicites. |
| trap       | Empreinte d'armement au sol, puis déclenchement par une cible admissible.                          | spawn, trigger, expire     | Délai d'armement, durée d'attente, filtre, nombre maximal de déclenchements.          |

La composition temporelle provient des relations entre ces porteurs. Une fragmentation est un enfant projectile lancé sur `hit` ou `expire` ; une explosion est un enfant pulse ; une zone persistante est un enfant field ; un piège peut déclencher n'importe lequel d'eux. Il n'est pas nécessaire d'ajouter un « sort météore » ou un « sort mine de feu » dans le code pour réaliser ces assemblages.

## Paramètres communs

Chaque nœud contient `node_id`, `subject_id`, `clause_ids`, `carrier`, `activation`, `anchor`, `geometry_id`, `scale_cm`, `rotation_mdeg`, `effects` et ses options typées. L'ancrage vaut `caster`, `aim_point` ou `parent_vent`. L'orientation est celle du lancement, sauf règle contraire explicite du porteur. Une zone ne suit donc pas automatiquement la caméra ou le curseur après sa création.

Les unités autorisées sont les centimètres pour les distances, les ticks pour les durées, les milli-degrés pour les angles et les milli-points pour santé et structure. Un tick vaut 20 ms. Une impulsion est stockée en milli-newton-secondes ; Unity la convertit en N.s. Les entrées JSON mécaniques utilisent des entiers ; la conversion en flottants intervient uniquement aux frontières du moteur.

## Filtres de cible

`hostile`, `ally`, `self`, `all_actors`, `environment`. `ally` exclut le lanceur ; `self` le désigne uniquement ; `all_actors` inclut le lanceur. Les filtres découlent d'un service de relation et non de la couleur du matériau. Les objets physiques non personnages peuvent recevoir `impulse` ou des statuts seulement s'ils possèdent le composant récepteur requis.

Les collisions matérielles et le ciblage sont distincts. Un projectile qui ne soigne que les alliés peut être arrêté par un mur. Une barrière arrête les projectiles matériellement ; elle ne choisit pas de laisser passer un ennemi parce que son effet cible seulement des alliés.

## CHAPITRE 10 / SP-1.0

# Du dessin à la géométrie 3D

## Préserver une signature au-delà de la couleur

Un sort ne conserve pas son dessin simplement parce que sa vignette est projetée sous une boule générique. Chaque paquet doit référencer au moins une géométrie issue de ses pixels utilisée dans la trajectoire, l'empreinte, la répartition des émissions ou la silhouette principale. Une signature décorative supplémentaire peut utiliser le dessin entier.

Le résolveur ne cherche pas « quel symbole signifie quoi ». Il transforme des marques visibles en données géométriques utilisables. La décision que la couronne représente une trajectoire vient de A ; l'extraction de cette trajectoire depuis l'encre est une opération géométrique.

## Banque de géométries

Pour chaque région demandée et pour le dessin entier, conserver le masque d'alpha, sa boîte englobante, ses composantes connexes, ses contours et un chemin principal lorsque celui-ci existe. Les identifiants sont stables dans la capture : `ring.footprint.0`, `outer.path.0`, etc. Chaque entrée indique le hash du masque source, la région, son algorithme et ses tolérances.

Le nettoyage technique élimine uniquement les artefacts numériques sous le seuil documenté. Il ne referme pas arbitrairement des trous, ne symétrise pas et ne fusionne pas les pointillés pour « corriger » un dessin. Paramètres proposés : masque à  $512^2$  dérivé de l'original  $1\,024^2$ , seuil d'alpha 16/255, simplification maximale 1 pixel du masque de travail, conservation des composantes ayant au moins 2 pixels. Ces seuils sont soumis à revue sur les dessins fins.

## Trois résolutions différentes

**Empreinte** : l'autorité de contact d'une zone est un masque 2D versionné dans son rectangle local, avec une hauteur 3D explicite. Un broadphase physique collecte les récepteurs proches ; le narrowphase projette leurs volumes dans le masque. La présence de trous et de fragments demeure mesurable. La bordure visible est dérivée du même masque.

**Chemin** : squelettiser le masque sans inventer de branche ; sélectionner un chemin principal déterministe, puis le rééchantillonner en 16 à 128 points. Les branches restantes peuvent servir à la répartition ou à la décoration si A le demande. Si plusieurs chemins ont la même longueur, utiliser l'ordre stable des pixels, pas l'ordre du dictionnaire. Les passages quasi nuls sont signalés ; une région vide ne devient pas silencieusement une ligne droite.

**Barrière** : construire des bandes/segments extrudés à partir du chemin ou du contour retenu. Les colliders composés de capsules ou de boîtes suivent les mêmes segments que le mesh. Les segments très courts sont fusionnés avec une tolérance déclarée. Pas de `MeshCollider` concave animé produit librement par le modèle.

## Transformation dans le monde

Chaque géométrie utilise un repère local centré, avec son ratio largeur/hauteur conservé. Pour une empreinte au sol : axe image horizontal → axe droit du lanceur ; axe image vertical inversé → axe avant ; hauteur → axe vertical Unity. La taille du plus grand côté est `scale_cm` ; elle ne dépend pas de la résolution d'affichage du parchemin.

Pour une trajectoire, normaliser le chemin entre son début et sa fin ; une paramétrisation par longueur d'arc fournit une vitesse régulière. Si le chemin est fermé, le point de départ est le pixel admissible le plus haut puis le plus à gauche ; la règle est conservée dans l'asset. La courbe oriente le déplacement mais la collision utilise toujours la position précédente et la suivante. Une fois le chemin courbe entièrement parcouru, le projectile continue sur sa tangente finale jusqu'à sa portée, sa durée ou un arrêt matériel ; il ne reboucle pas. Le mode guidé utilise le point de départ et la direction initiale fournis, puis corrige son orientation vers le récepteur acquis dans la limite d'angle par tick ; il ne prétend pas suivre simultanément toute la courbe. Ce choix évite deux trajectoires contradictoires.

## Désaccord géométrique

Si le masque choisi est vide ou inexploitable pour le rôle requis, le résolveur renvoie une erreur explicite avec les alternatives géométriques réellement disponibles. B peut reformuler sa construction avec une géométrie de la même description, mais ne remplace pas une zone dessinée par une sphère sans provenance. Une adaptation de simplification est enregistrée dans `geometry_report` et visible dans la revue concepteur.

La silhouette visuelle peut comporter des détails plus fins que les collisions, mais la zone d'effet principale ne doit pas tromper : cible maximale proposée, écart de frontière inférieur à 10 cm au sol et inférieur à 5 % de la taille du sort, prendre la plus stricte des deux. Pour un dessin trop fin pour ces seuils, exposer l'incertitude dans la revue ; ne pas affirmer une fidélité parfaite.

## CHAPITRE 11 / SP-1.0

# Effets, physique et réactions

## Six effets explicitement programmés

L'affinité colore la matière et peut orienter l'interprétation ; elle ne déclenche aucune capacité non déclarée. Un porteur de feu ne brûle que si un effet `burn` ou une règle explicite le prévoit. Un porteur d'eau ne soigne pas automatiquement. Cette séparation permet des usages variés sans inventer un rôle garanti par le pinceau.

| Effet                | Paramètre amount                                | Exécution normative                                                                                         |
|----------------------|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <code>damage</code>  | Milli-points de santé ; 1 000 = 1 point.        | Retrancher à la santé, bornée à zéro ; la mort est signalée au banc d'épreuve.                              |
| <code>heal</code>    | Milli-points de santé.                          | Ajouter sans dépasser la santé maximale ; ne ressuscite pas une cible morte.                                |
| <code>impulse</code> | Milli-N·s ; 1 000 = 1 N·s.                      | Appliquer une impulsion à un corps dynamique ; sur acteur piloté, passer par son adaptateur de déplacement. |
| <code>burn</code>    | Milli-points par tick de brûlure d'une seconde. | Premier dégât après 50 ticks ; durée multiple de 50 ticks ; aucune propagation implicite.                   |
| <code>wet</code>     | Doit être zéro.                                 | Appliquer l'état Mouillé pendant la durée ; participe seulement aux réactions définies ci-dessous.          |
| <code>slow</code>    | Réduction en millièmes ; 300 = -30 %.           | Modifier le facteur de vitesse de l'acteur, jamais la cadence globale du jeu.                               |

Les paramètres nuls sont autorisés pour dégâts/soin/impulsion : un résultat faible n'est pas systématiquement une erreur technique. Une liste d'effets vide peut représenter un phénomène visuel assumé par A. Le worker ne doit cependant jamais créer un tel phénomène pour masquer une API en panne.

## Santé et structure

Le laboratoire utilise des cibles à 100 points de santé, dont certaines commencent à 50. Les valeurs de dégâts des exemples servent à mesurer la boucle, pas à équilibrer le futur jeu. Les barrières possèdent une structure séparée. Aucun soin de barrière n'est admis sans effet dédié dans une version ultérieure. SP-1.0 traite la structure d'une barrière comme un récepteur d'environnement : seuls les effets `damage` filtrant `environment` l'endommagent. Les autres projectiles sont arrêtés mais ne réduisent pas sa structure. Le texte de règle doit mentionner cette distinction lorsque pertinente. Une future règle universelle d'usure serait un changement de contrat.

## Statuts et cumul

Pour une même cible et une même famille de statut, conserver la plus forte magnitude et l'expiration la plus tardive ; les applications ne s'additionnent pas. Une magnitude plus faible peut prolonger la durée, sans abaisser la magnitude en cours. À expiration, le statut disparaît ; on ne restaure pas une ancienne version plus faible. À égalité, l'attribution à la source la plus ancienne est conservée.

Le ralentissement est plafonné à 75 %. La brûlure ne provoque pas elle-même de nouveaux événements `hit`, de fragmentation ni de chaîne : ses dégâts sont des événements de statut, pas des impacts de porteur. Cela coupe les boucles involontaires d'auto-propagation.

## Deux réactions matérielles livrées

**Mouiller une cible brûlante** : supprimer la brûlure puis appliquer Mouillé. **Enflammer une cible mouillée** : consommer Mouillé sans appliquer cette brûlure-là ; une application ultérieure pourra brûler. Un bref effet de vapeur signale la réaction, sans dégâts additionnels. Les réactions sont ordonnées et testées ; les événements simultanés utilisent l'ordre stable décrit au chapitre 13.

Les objets de test exposent des capacités : `HealthReceiver`, `StatusReceiver`, `ImpulseReceiver`, `Burnable` et `SolidObstacle`. Un objet n'ayant pas le récepteur nécessaire n'acquiert pas une propriété parce que le modèle l'a évoquée. Une caisse est réellement déplacée par l'impulsion ; une simple animation qui la fait vibrer ne valide pas la mécanique.

## Couche physique Unity

Utiliser des volumes de collision explicites et des couches : `WorldSolid`, `Actor`, `DynamicProp`, `SpellBarrier`, `SpellVisual`, `Ground`. Les VFX n'interviennent jamais dans les collisions. Les corps décoratifs sans récepteur ne reçoivent pas d'effets métier.

Pour les projectiles, balayer le volume de la position précédente à la suivante. Une simple recherche de collision à la position d'arrivée manquerait un mur mince à grande vitesse. Une requête non allouante peut servir, mais son résultat n'est pas implicitement trié et un tampon plein exige un traitement explicite ; ne jamais prendre l'élément zéro pour le premier impact. [W11]

Traiter d'abord les chevauchements au point de départ, puis le balayage. Trier par distance puis identifiant de récepteur stable. Réunir les colliders appartenant à la même cible avant application des effets. Le caster est exclu du contact initial tant que le projectile n'a pas quitté son volume, selon un rayon de sortie enregistré ; cela ne supprime pas un futur retour sur soi lorsque le filtre l'autorise.

## CHAPITRE 12 / SP-1.0

# Compilation, bornes et paquet immuable

## Ordre de compilation obligatoire

1. Contrôler les octets : taille, profondeur JSON, clés dupliquées, type et version. Appliquer le schéma strict et refuser les identifiants inconnus.
2. Résoudre les références : clauses, sujets, parents, géométries, profils de matériau et versions du catalogue. Détecter les cycles et les nœuds inatteignables.
3. Vérifier les compatibilités : options par porteur, événements disponibles, filtres, unités, géométrie et effets annoncés par la description.
4. Calculer les bornes pessimistes : instances, durée totale incluant les statuts résiduels après disparition des porteurs, événements, contacts, géométrie et mémoire. Un dépassement retourne une erreur détaillée à B.
5. Établir l'ordre stable, les paramètres finaux, la signature et les données géométriques référencées. Produire la fiche de règles depuis ces données.
6. Sérialiser une fois, calculer l'empreinte du payload et enregistrer le résultat dans une transaction de publication. Ne plus relancer A ou B pour ce support.

## Bornes proposées du profil `lab_v1`

| Ressource                                  | Limite du profil       | Statut                                       |
|--------------------------------------------|------------------------|----------------------------------------------|
| Nœuds dans un plan                         | 16                     | Borne technique, pas nombre de sorts du jeu. |
| Copies créées par activation               | 8                      | Les émissions multiples restent bornées.     |
| Activations d'un nœud enfant par lancement | 8                      | Compteur global par nœud et par lancement.   |
| Instances créées sur la vie d'un lancement | 128                    | Calculé avant publication.                   |
| Durée d'une instance                       | 500 ticks, soit 10 s   | Selon le porteur.                            |
| Dernier événement possible d'un lancement  | 1 500 ticks, soit 30 s | Inclut délais et enfants.                    |
| Cibles distinctes par instance             | 32                     | La fiche expose les limites pertinentes.     |
| Applications d'effets par lancement        | 8 192                  | Borne pessimiste du compilateur.             |
| Points d'un chemin de collision            | 128                    | Avec simplification et rapport de fidélité.  |
| Colliders d'une barrière                   | 64                     | Composés et préalloués.                      |

| Ressource                | Limite du profil                 | Statut                                  |
|--------------------------|----------------------------------|-----------------------------------------|
| Portée / taille maximale | 30 m / 12 m                      | Paramètres de laboratoire.              |
| Lancements simultanés    | 8, sous réserve du budget global | Admission avant création des instances. |

Les limites techniques ne doivent pas servir à rééquilibrer silencieusement un sort publié. Le plan de B est construit dans ces limites. Un dépassement provoque une réparation avant publication, jamais une modification cachée de la description ensuite.

## Calcul borné des compositions

`activation.max_activations` est un compteur **par nœud et par lancement complet**, partagé par toutes les instances parentes. Il ne se réinitialise pas à chaque projectile. Pour une racine, il vaut 1. Le nombre pessimiste d'instances d'un nœud est `copies × max_activations` ; la somme doit rester inférieure ou égale à 128. Cette convention évite les explosions combinatoires involontaires.

La durée maximale est évaluée le long de tous les chemins du graphe. Une borne sûre additionne, sur un chemin, délai d'activation et durée maximale de chaque porteur. Même si un impact intervient souvent avant l'expiration, il ne faut pas compter sur cette probabilité pour valider le plan.

Les contacts d'un projectile sont bornés par ses pénétrations, rebonds et durée. Un faisceau est borné par sa cadence et ses sauts. Une zone applique au maximum 32 effets de contact à chaque tick prévu. Un piège possède un nombre de déclenchements. Les applications d'effets incluent les ticks de statut et leurs durées, pas seulement les hits immédiats. Le compilateur utilise des entiers 64 bits et des additions/multiplications vérifiées ; un dépassement arithmétique est une erreur.

Une barrière émet au maximum 32 événements `block` sur sa vie, puis expire ; cette limite protège les enfants déclenchés par blocage et doit figurer dans les règles du porteur. Les effets de blocage ne reçoivent pas automatiquement comme cible le propriétaire lointain du projectile : ils ciblent le récepteur au point de contact lorsqu'il existe, sinon seul un enfant peut créer une zone autour du contact.

## Budget global au lancement

Avant d'accepter un cast, réserver son budget pessimiste de porteurs et de travaux. Lorsque plusieurs casts remplissent le budget global, afficher « Trop de sorts actifs » et ne pas démarrer celui-ci. Ne pas lancer visuellement le sort puis retirer la moitié de ses effets. Les réservations sont libérées à l'extinction du dernier événement du cast.

Un garde-fou runtime subsiste contre les bugs : si un événement dépasse les bornes compilées, arrêter le cast, enregistrer un incident `runtime_budget_violation` et le signaler dans la revue. Ce mécanisme n'est pas une dégradation normale de qualité et ne doit pas être masqué.

## Identité et hash

`parchment_id` identifie le support ; `job_id` le travail technique ; `spell_id` le sort publié ; `cast_id` un lancement. Ces identifiants ne sont jamais interchangeables. L'empreinte de l'image sert à vérifier le contenu, pas à forcer deux supports identiques à partager un sort.

Le paquet contient les versions `schema`, `catalog`, `compiler`, `geometry`, `rules_profile` et les identifiants de prompts/modèles utilisés. Les valeurs aléatoires nécessaires sont dérivées d'une graine de signature attribuée au support puis conservées. Les paramètres mécaniques finaux sont déjà numériques et ne sont plus tirés au lancement.

Pour simplifier la vérification entre C# et Unity, le serveur fournit les octets JSON UTF-8 du payload compilé et leur SHA-256 ; le client vérifie les octets reçus avant parsing. Le hash détecte une corruption, **pas une falsification par un attaquant**. La provenance distante repose sur HTTPS et l'autorisation du service dans ce périmètre de laboratoire. Un futur mode compétitif nécessiterait une autorité de simulation et un modèle anti-triche distincts.

**CHAPITRE 13 / SP-1.0**

# Exécution Unity : classes, événements et cycle de vie

## Objets centraux

`SpellRuntime` valide le paquet et expose `TryCast`. `CastContext` contient l'acteur source, la position, le point visé, la direction, le tick et l'identifiant de cast. `SpellScheduler` distribue les activations. `CarrierPool` réutilise les objets des six familles. `EffectDispatcher` transmet les effets aux récepteurs. `StatusSystem` suit les statuts indépendamment des VFX. `PresentationBridge` observe les événements sans décider des règles.

L'utilisation d'un pool est recommandée pour ne pas recréer toutes les instances à chaque lancement. Le pool standard de Unity fournit une base, mais les règles de remise à zéro sont à programmer et à tester pour ce jeu. [W12]

## Ordre d'un tick de simulation

Le pas de simulation est 20 ms. Les délais sont des nombres de ticks. L'ordre logique proposé est : demandes de lancement acceptées ; activations planifiées ; mouvement et détection des contacts ; tri/déduplication ; effets et réactions ; événements enfants ; expirations ; libération des instances. Les créations d'enfants sont planifiées au plus tôt au tick suivant, même avec un délai zéro, afin d'éviter une récursion dans la pile du même tick.

Un événement possède au minimum `tick`, `cast_id`, `node_id`, `instance_id`, `event_kind`, `event_sequence`, `position`, `normal` et `receiver_id` nullable. Les événements d'un tick sont triés par ces identifiants stables dans cet ordre logique. Les effets d'un nœud suivent leur ordre d'identifiant. Le `parent_event` est un instantané : un enfant ne dépend pas de l'existence ultérieure du `GameObject` parent.

Le noyau assure un ordre logique reproductible. Unity Physics peut présenter des écarts entre plateformes ; la recette n'exige pas de physique bit à bit multiplateforme. Les tests comparent des règles exactes sur un monde simulé déterministe et des tolérances géométriques dans la scène Unity.

## Projectile

À la création, placer le porteur, initialiser sa trajectoire et vider toutes les collections de contact. À chaque tick, intégrer le déplacement, balayer son volume et traiter les contacts dans l'ordre. Les pénétrations s'appliquent aux acteurs ; les rebonds aux surfaces solides. Ne pas déclencher rebond et pénétration sur le même contact. Après un rebond, décaler légèrement la position le long de la normale pour éviter un contact immédiat répété.

Le guidage recherche une cible du filtre annoncé, dans la portée et la ligne de vue. La cible la plus proche gagne, puis l'identifiant stable en cas d'égalité. Si elle devient invalide, effectuer une nouvelle recherche bornée. Sans cible, le projectile poursuit sa trajectoire ; il ne disparaît pas et ne se téléporte pas. Une vitesse maximale et une rotation maximale par tick sont appliquées aux valeurs déjà compilées.

## Faisceau et chaîne

Un faisceau instantané ne dure qu'un tick ; un faisceau entretenu évalue à sa cadence `tick_interval`. Chaque segment s'arrête sur le premier obstacle. Une chaîne recherche un nouveau récepteur à partir du précédent, sans revisiter une cible du même parcours ; chaque saut doit avoir une ligne de vue. Le nombre de sauts signifie « cibles supplémentaires après la première ». La chaîne n'augmente pas les dégâts sauf paramètre déclaré ; aucun multiplicateur caché.

## Zone, onde et piège

Une zone effectue broadphase puis test dans le masque. `enter` est émis une seule fois par récepteur et par instance dans SP-1.0, même après sortie et retour ; `tick` continue à la cadence annoncée pendant la présence. Le premier `tick` intervient à `spawn_tick + tick_interval`, pas immédiatement. Les effets de `spawn` utilisent un contexte de zone uniquement lorsqu'un porteur le définit explicitement.

Une onde balaie l'espace entre son front précédent et le nouveau, plutôt que de tester seulement une frontière instantanée. Elle touche chaque cible une fois. Un piège ignore les cibles avant `arm_ticks`, déclenche à l'entrée admissible, puis respecte son délai de réarmement et son maximum de déclenchements. Si une cible est déjà présente au moment de l'armement, le tick d'armement déclenche une fois.

## Barrière et extinction

La barrière dispose de colliders solides, d'une structure et d'un maximum de blocages. À sa destruction ou expiration, elle cesse immédiatement de bloquer puis son VFX de fin peut se poursuivre. Les projectiles ne sont pas bloqués par un renderer invisible resté dans le pool.

À la restitution au pool, vider récepteurs visités, abonnements, références au propriétaire, timers, états physiques, propriétés de matériau et buffers de particules. Une instance rendue au pool ne peut émettre `expire` une seconde fois. Fermer la scène annule ses casts et tâches de simulation, pas les tâches de génération serveur.

## Interfaces d'intégration à fournir

```
public interface ISpellRuntime
{
    CastResult TryCast(CompiledSpell spell, CastContext context);
    void Tick(int tick);
    void CancelAll(SceneScope scope);
}

public interface ISpellWorld
{
    // Les résultats sont bornés, identifiés et sans doublon de récepteur.
    QueryResult Sweep(SweepRequest request, ContactBuffer contacts);
    QueryResult QueryArea(AreaRequest request, ReceiverBuffer receivers);
    void Apply(EffectCommand command);
}
```

Ces signatures sont des contrats d'architecture à réaliser, pas une bibliothèque Unity déjà fournie. Les tests du noyau utilisent un `ISpellWorld` déterministe ; l'adaptateur Unity gère les layers, les colliders et les Rigidbody. Aucun appel physique ne doit être caché dans la couche visuelle.

**CHAPITRE 14 / SP-1.0**

# Rendu, son et qualité de présentation

## Une bibliothèque de matières, pas de sorts préfabriqués

Les six porteurs disposent chacun d'un renderer générique. Quatre profils de matière sont nécessaires : braise, eau, pierre et souffle. Un profil précise transparence, couleur, émission, texture de bord, vitesse de distorsion et famille sonore. Les formes viennent des géométries du dessin ; les matériaux donnent leur matière. Un `FireballPrefab` complet ne doit pas être le résultat universel du système.

La création finale n'utilise pas d'image ou de shader généré par le modèle. Les shaders sont écrits et testés pendant le développement, avec une liste finie de paramètres. Les textures de signature issues du dessin sont des données bornées. Les références d'assets viennent du catalogue installé, pas d'une URL produite par B.

## Correspondance forme / collision

La trajectoire principale, l'épaisseur utile, l'empreinte au sol et l'orientation proviennent des mêmes données que la simulation. Les particules secondaires peuvent dépasser légèrement, mais elles ne suggèrent pas une zone de dégâts deux fois plus large. Le mode concepteur superpose frontière physique et frontière visible pour vérifier le contrat.

Le style du trait doit se retrouver dans au moins deux propriétés perceptibles lorsque le dessin le permet : un double tracé donne deux filaments visuels distincts ; une trace fragmentée garde des interruptions ; un contour asymétrique ne devient pas un cercle parfait. Ce sont des objectifs de rendu guidés par l'interprétation, non des tables garantissant un effet tactique particulier.

## Chronologie et finitions

Chaque porteur dispose d'une naissance, d'une phase active et d'une extinction. Le délai visuel ne doit pas décaler secrètement la collision : un télégraphe précède le tick d'activation et n'applique pas encore d'effets. Un impact émet son flash au contact réel. Un piège montre son armement en mode laboratoire ; son invisibilité éventuelle dans le futur jeu n'est pas décidée ici.

L'audio possède des couches de création, lancement, boucle et contact, avec durée contrôlée et arrêt propre. Les sons d'impact suivent les événements physiques, pas une animation prédéterminée. Des variations légères de timbre ou de phase utilisent la signature du parchemin ; elles ne changent pas à chaque chargement. Toutes les sources d'assets et licences sont consignées.

## Barrière de qualité humaine

La version finale ne peut pas se limiter à des sphères grises et à des sons manquants sous prétexte que les nombres fonctionnent. L'équipe valide la lisibilité, la cohérence du matériau, le lien au dessin et la sensation de lancement pour chaque famille. Un rendu élégant mais déconnecté de ses collisions échoue ; un runtime exact mais sans présentation terminée échoue également.

## CHAPITRE 15 / SP-1.0

## Scénarios de référence et exemple complet

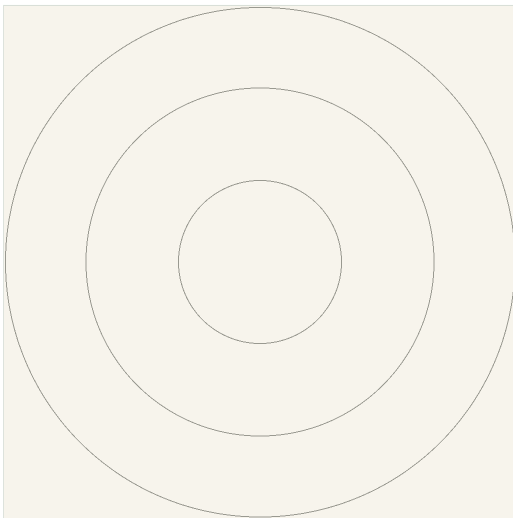


Image 1 · Référence du layout

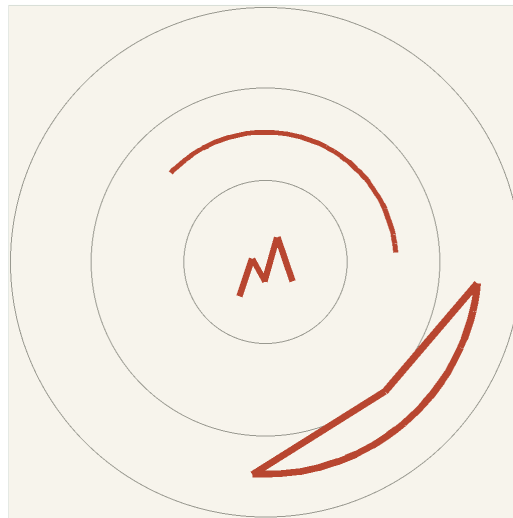


Image 2 · Dessin synthétique

Entrées illustratives. Les fixtures JSON sont rédigées manuellement ; elles ne sont pas des réponses de modèle ni des résultats Unity.

Les exemples ci-dessous sont **des cas de conception**, pas des résultats réels de modèle et pas des recettes de joueur. Ils servent à vérifier des comportements précis. La réussite visuelle d'un futur appel ne se déduit pas de leur présence dans ce dossier.

### Cas principal — Projectile courbe puis empreinte brûlante

Le dessin de test comporte une marque de braise au noyau, une trace courbe dans la couronne et une marque de persistance sur la périphérie. À interprète : « Une braise suit une courbe ; au premier contact avec un adversaire, elle laisse une empreinte brûlante persistante. » Les observations doivent réellement correspondre aux marques.

B crée un projectile `p0`, filtrant les adversaires, puis un champ `f1` déclenché par `p0.hit`, avec une activation maximale de 1. Le projectile inflige 12 points au contact. La zone inflige une brûlure de 2 points par seconde pendant 3 secondes aux adversaires présents, et rafraîchit ce statut à sa cadence sans cumuler les magnitudes. Les géométries proviennent de la couronne pour le chemin et de la périphérie pour l'empreinte.

Les nombres sont illustratifs et enregistrés dans les fixtures livrées. Les critères sont : le projectile suit le chemin, ne traverse pas le mur, touche une cible une fois, fait apparaître le champ au premier impact hostile (pas sur un mur ou un allié), le champ ne touche pas une cible située dans un trou de son masque, le statut est mesurable, l'ensemble s'éteint et le même paquet fonctionne après redémarrage.

## Cas de contraste — Même matière, fonction différente

Un second dessin interprété comme une onde d'eau réparatrice doit produire `pulse + heal`, et non des dégâts automatiques parce qu'un autre sort d'eau en produisait. Une cible alliée blessée est soignée ; une cible ennemie ne l'est pas ; une caisse ne reçoit pas une santé fictive. Le filtre de la description et celui du runtime doivent correspondre.

## Cas de composition — Piège, fragmentation et chaîne

Un piège peut déclencher plusieurs projectiles à l'approche d'un adversaire ; un faisceau peut se relayer entre plusieurs récepteurs sans revisiter une cible. Ce sont deux tests séparés avant un test composé. Chaque nombre d'enfants, délai, cible et saut doit apparaître dans le plan et dans le calcul de budget.

## Cas d'émergence physique limitée

Une onde de souffle pousse deux caisses de masses différentes. Le déplacement résulte de l'impulsion et de la physique, pas d'un scénario animé. Un autre sort applique Mouillé à une cible brûlante et éteint le statut. Ces tests prouvent une interaction avec le monde dans le périmètre livré, sans prétendre déjà réaliser un moteur universel d'éléments.

## Cas négatif de gameplay

Un gribouillage produit un champ très court et peu intense, mais cohérent avec l'interprétation retenue. Le parchemin reste tel quel. Ce résultat peut être peu utile ; ce n'est pas une raison pour afficher une gomme ou proposer de relancer la génération. À l'inverse, une erreur HTTP ou un effet inconnu n'est pas un « sort raté » acceptable.

## CHAPITRE 16 / SP-1.0

# API du service de génération

Le service s'exécute séparément de Unity. Le client connaît une URL HTTPS et un jeton d'accès du laboratoire, jamais la clé du fournisseur. La description de l'API est livrée dans `contracts/openapi.yaml`. Les identifiants et l'appartenance des ressources sont vérifiés côté serveur ; un `owner_id` fourni par le client n'est pas une autorité.

| Route                                        | Rôle                                                                 | Réponse attendue                                      |
|----------------------------------------------|----------------------------------------------------------------------|-------------------------------------------------------|
| GET <code>/v1/capabilities</code>            | Versions, layout, encres et capacités du build.                      | 200, manifeste de compatibilité.                      |
| GET <code>/v1/parchments</code>              | Liste paginée des supports du compte de test.                        | 200, maximum 50 résultats et curseur.                 |
| POST <code>/v1/parchments</code>             | Allouer un support vierge du laboratoire.                            | 201 ; même clé d'idempotence → même allocation.       |
| POST <code>/v1/parchments/{id}/begin</code>  | Marquer la première inscription durable.                             | 200 ; doublon sans nouvelle consommation.             |
| PUT <code>/v1/parchments/{id}/capture</code> | Transmettre la capture finale ; déclencher automatiquement la tâche. | 202 avec <code>job_id</code> et version de ressource. |
| GET <code>/v1/jobs/{id}</code>               | Lire l'étape réelle, les tentatives et le résultat.                  | 200 ; aucun secret ou prompt interne complet.         |
| POST <code>/v1/jobs/{id}/resume</code>       | Reprendre un incident technique autorisé.                            | 202 ; même dessin et même identité de travail.        |
| GET <code>/v1/spells/{id}</code>             | Récupérer les octets du paquet publié.                               | 200 ; X-Content-SHA256 pour l'intégrité.              |
| GET <code>/v1/artifacts/{id}</code>          | Récupérer un PNG ou une géométrie autorisée.                         | 200 ; type et hash dans le manifeste.                 |
| POST <code>/v1/reviews</code>                | Enregistrer une revue explicitement soumise par un humain.           | 201, revue immuable ; corrections par nouvelle revue. |
| POST <code>/v1/authoring/plan</code>         | Tester B avec une description rédigée par un concepteur.             | 202, tâche de diagnostic, hors inventaire joueur.     |

## Capture et idempotence

L'upload est multipart : `capture` contient le manifeste JSON, `drawing` le PNG complet, `ink` le masque PNG et `journal` le journal compressé. L'image de référence est identifiée par sa version ; le serveur possède les octets autorisés et ne fait pas confiance à une image de référence remplacée par le client.

L’empreinte de requête porte sur le contenu normalisé du manifeste et les hashes des fichiers, **pas sur la boundary multipart**, qui peut changer lors d’une reprise. Une même clé et le même contenu retrouvent le résultat ; une même clé avec un contenu différent renvoie 409. Un support ayant déjà une capture finale accepte uniquement une transmission identique. Il n’existe pas de route de remplacement du dessin.

L’upload complet est vérifié avant de créer la tâche. Une écriture temporaire suivie d’un renommage atomique rend les fichiers durables ; une transaction PostgreSQL attache la capture et crée le job. En cas de panne entre stockage et transaction, les fichiers orphelins sont nettoyés après délai, sans déclarer le dessin publié. La tâche n’est jamais créée avec un fichier encore en cours d’envoi.

## États de tâche

`queued` → `interpreting` → `resolving_geometry` → `planning` → `validating` → `ready`. `waiting_retry` et `needs_operator` sont des états d’incident latéraux, avec `resume_stage`, compteur de tentatives et dernier code d’erreur. Une sortie de A déjà persistée est réutilisée lors d’une reprise de B. Une tâche `ready` est terminale ; `resume` y retourne le résultat existant sans appel fournisseur.

Le client interroge l’état toutes les 2 secondes au départ, puis espace jusqu’à 10 secondes. Il respecte les temporisations serveur. Fermer l’application arrête ce polling, pas le travail du service. La réouverture retrouve le job par le support, même si la réponse au premier upload s’était perdue.

## Erreurs visibles et erreurs internes

Toutes les erreurs HTTP renvoient un objet comprenant `code`, `message`, `trace_id` et `retryable`. Utiliser 400 pour un format incorrect, 401/403 pour l’accès, 404 pour une ressource inaccessible, 409 pour un conflit d’identité, 413 pour une taille dépassée, 422 pour une incompatibilité technique de capture et 429 pour une limite d’usage. Les sorties fautives de modèle sont des incidents de tâche, pas des erreurs de dessin imputées au joueur.

Les messages utilisateur distinguent « Transmission en attente », « Service temporairement indisponible », « Construction du sort à reprendre » et « Capture endommagée ». Ils n’affichent ni clé, ni trace interne, ni données d’un autre utilisateur. Un texte modéré/refusé par le fournisseur ne déclenche pas une boucle de tentatives de contournement ; le cas est arrêté et présenté comme incident adapté.

## CHAPITRE 17 / SP-1.0

# Persistence, reprise et unicité

## Tables à réaliser

| Entité            | Données essentielles                                                     | Contrainte critique                       |
|-------------------|--------------------------------------------------------------------------|-------------------------------------------|
| parchments        | propriétaire, état, layout, budget, première inscription, capture finale | Une capture finale au plus par support.   |
| captures          | hashes, fichiers, journal, version raster, raison de clôture             | Contenu immuable une fois attaché.        |
| jobs              | étape, tentatives, lease, fencing token, prochaine reprise               | Un job de production par capture.         |
| provider_attempts | étape, configuration, statut, latence, usage, réponse brute protégée     | Toutes les tentatives sont traçables.     |
| interpretations   | description A, hash, prompt et modèle                                    | Ne pas écraser pour faire correspondre B. |
| geometry_assets   | masques, chemins, rapports et sources                                    | Références valides dans le paquet final.  |
| spell_plans       | recette, erreurs, tentatives de réparation                               | Les révisions restent consultables.       |
| spells            | payload final, hash, profil, signature                                   | Une publication canonique par support.    |
| reviews           | auteur humain, build, cas, verdict et commentaire                        | Pas d'approbation automatique de l'agent. |
| idempotency_keys  | propriétaire, route, clé, empreinte, réponse                             | Clé isolée par propriétaire et opération. |

Stocker les métadonnées dans PostgreSQL et les fichiers dans un volume privé derrière `IArtifactStore`. Pour un laboratoire sur une seule machine serveur, un stockage filesystem durable suffit ; l'interface doit permettre un stockage objet ultérieur sans changer les contrats. Aucun répertoire d'upload ne doit être exposé comme site statique public.

## Worker robuste

La file est persistée dans PostgreSQL. Un worker prend un job avec un verrou bref, par exemple `FOR UPDATE SKIP LOCKED`, puis relâche la transaction avant l'appel distant. Ce mécanisme est documenté pour éviter d'attendre sur des lignes déjà verrouillées ; il faut programmer le lease et la reprise autour de lui. [W13]

Le lease proposé dure 180 secondes et est renouvelé toutes les 10 secondes. Chaque acquisition augmente un fencing token ; une réponse provenant d'un ancien worker ne peut plus publier. Les erreurs réseau et délais du fournisseur sont comptés et conservés. Un redémarrage du worker reprend à la dernière étape persistée.

Une transaction unique publie `spells` et passe le job en `ready`. Une contrainte unique empêche deux workers de publier deux sorts pour le même support. **Cela garantit l'unicité de publication interne, pas une facturation exactement une fois chez le fournisseur.** Un crash après réception distante mais avant persistance peut rendre nécessaire un nouvel appel ; ce coût doit être visible.

## Politique de tentatives

Valeurs proposées : 120 secondes de timeout HTTP par appel, deux réparations de schéma maximum par étape, deux reprises de transport maximum par étape et **dix requêtes fournisseur maximum pour le job entier**. Le plafond global prime sur les plafonds locaux. Après dépassement, `needs_operator`. `Retry-After` est respecté ; une attente n'augmente pas le cooldown du sort.

Les réservations de coût sont prises avant l'appel. Si l'état de facturation d'une tentative est incertain, conserver la réservation pessimiste jusqu'à réconciliation. Une nouvelle tâche ne doit pas saturer le budget en lançant des workers concurrents sur le même support.

## Créations proches et identité distincte

L'identité vient du support ; la sémantique vient du dessin. Il n'existe pas de cache « image identique = sort de quelqu'un d'autre ». Chaque nouvelle création reçoit une signature propre avant l'interprétation, puis le résultat est figé.

P05 limite les variations procédurales à la phase, la répartition secondaire, la texture et de faibles différences de silhouette à l'intérieur de la tolérance de collision. Le compilateur ne change pas aléatoirement les dégâts après A/B. Deux sorts peuvent rester mécaniquement proches, voire partager certains nombres. L'originalité de leur comportement doit venir de la richesse d'interprétation et de composition, pas seulement d'un UUID.

Ce choix n'est pas une preuve que deux humains ne produiront jamais des résultats visuellement identiques. La revue de paires au chapitre 21 doit vérifier que la signature ne se réduit pas à un changement de nom. Une exigence future d'unicité mécanique absolue nécessiterait un mécanisme supplémentaire et une décision explicite.

## Compatibilité au chargement

Un paquet indique ses versions minimales de runtime et de catalogue. Une version inconnue n'est pas interprétée « au mieux » : le client demande une mise à jour ou affiche une incompatibilité. Les anciens paquets gardent leur profil de règles. Une migration est un traitement explicite conservant l'ancien payload et un rapport de changement ; elle ne rappelle pas A pour réinventer le dessin.

**CHAPITRE 18 / SP-1.0**

# Sécurité, secrets et coût

## Frontières de confiance

Le PNG, son journal, le texte de A et le JSON de B sont tous des entrées non fiables. Le modèle n'est jamais autorisé à choisir une URL de stockage, un chemin fichier, un type C# à charger, un nom de shader hors catalogue ou une commande. Toute référence est résolue via un identifiant autorisé.

Limiter les uploads : PNG uniquement pour les images, 1 024<sup>2</sup> pixels attendus, dimensions et type réels contrôlés après décodage, 8 Mio par image et 16 Mio pour la requête complète ; journal décompressé plafonné à 8 Mio et nombre de points plafonné à 65 536. Refuser les archives arbitraires. Les chemins internes sont générés côté serveur et ne reprennent pas le nom envoyé par le client.

Le client ne transmet pas ses clés de fournisseur. Le secret du fournisseur est installé uniquement dans l'environnement du backend ou un secret de déploiement. La documentation API recommande expressément de ne pas exposer les clés dans un client. [W14]

## Accès au laboratoire

Pour ce périmètre privé, provisionner des jetons aléatoires de test par une commande d'administration locale ; stocker leur empreinte côté serveur et autoriser leur révocation. Le token du client est conservé dans un stockage protégé du système Windows, jamais en clair dans un ScriptableObject ou dans Git. Un rôle `creator` est requis pour le mode texte et l'envoi de revues humaines.

HTTPS est obligatoire hors `localhost`. Les routes contrôlent l'appartenance de chaque ressource. Le service renvoie 404 pour un identifiant qui appartient à quelqu'un d'autre afin de ne pas révéler son existence. Ce dispositif n'est pas un système de compte public destiné à un MMO ; ne pas construire d'inscription publique pour finir cette boucle.

## Budget de génération

Les plafonds existent à trois niveaux : par utilisateur, par job et pour le service entier. La limite générale de débit est proposée à trois créations simultanées par utilisateur et quatre appels fournisseur simultanés au total pour le laboratoire. Les valeurs sont configurables ; elles ne constituent pas une promesse de capacité pour un serveur précis.

Le coût d'un job est la somme des usages facturés de A, B et leurs réparations, avec le tarif du modèle enregistré à la date de l'appel. Les images consomment un budget d'entrée ; ne pas compter uniquement le texte. [W8] La formule et les métriques sont implémentées, mais aucun coût réel par parchemin n'est affirmé avant mesure sur le corpus.

Une fois le budget du service épuisé, refuser les nouvelles tâches et conserver les captures engagées en attente, avec un message explicite. Ne pas basculer en mode factice. Ne pas changer de modèle vers un modèle moins coûteux sans versionner ce choix et faire accepter sa qualité.

## Logs et conservation

Les logs opérationnels contiennent les identifiants, étapes, durées, nombres de tokens/usage lorsqu'ils sont disponibles, versions et codes d'erreur. Les captures et réponses brutes sont privées, séparées des logs standards. Les journaux n'enregistrent jamais les tokens d'accès.

Politique technique proposée : dossiers de test conservés jusqu'à suppression par l'équipe, tentatives brutes limitées à 30 jours par défaut, métriques agrégées conservées plus longtemps sans images. Il s'agit d'un réglage de produit à adapter à l'usage, pas d'un avis juridique ni d'une déclaration de conformité. Les exports de revue avertissent lorsqu'ils incluent un dessin et une réponse brute.

## CHAPITRE 19 / SP-1.0

# Installation, exploitation et récupération

## Environnement de développement

Installer Unity Hub, l'éditeur verrouillé, le module Windows x64 IL2CPP, les composants de compilation Windows requis par cet éditeur, .NET SDK 10 et l'outil de conteneurs choisi pour le backend. Les prérequis exacts et versions sont consignés dans `docs/SETUP.md` au jalon M0 ; aucune installation manuelle oubliée ne doit être nécessaire au dernier moment.

Le dépôt doit fournir une commande de validation des contrats, une commande de tests .NET, une commande de lancement du backend et une méthode de build Unity reproductible. Les commandes ci-dessous sont le **contrat de remise à implémenter**, pas des commandes qui construisent un jeu depuis ce dossier de spécification :

```
dotnet test Palimpseste.sln
docker compose --env-file .env up -d --build
Unity.exe -batchmode -projectPath game -executeMethod Build.ReleaseWindows -quit
```

Les bibliothèques du cœur et les tests .NET sont construits sans lancer Unity. La compilation du client et les tests de scène s'effectuent avec le véritable éditeur verrouillé. La licence d'exécution de l'éditeur doit être disponible dans l'environnement de build ; ne pas présenter une commande échouant faute de licence comme une validation.

## Services attendus

Une composition locale comporte `api`, `worker`, `postgres` et, pour l'accès distant, un reverse proxy TLS. Les volumes de PostgreSQL et d'artefacts sont persistants. La base n'expose pas son port sur Internet. L'API possède `/health/live` et `/health/ready` ; le second vérifie ses dépendances internes sans lancer une génération facturée.

Le worker utilise la même image applicative que l'API ou un binaire distinct du même commit. Les migrations sont exécutées explicitement avant le démarrage applicatif, avec un mécanisme empêchant plusieurs migrations concurrentes. Le service n'exécute pas une migration destructive au premier appel d'un joueur.

## Configuration minimale

`DATABASE_URL`, `ARTIFACT_ROOT`, `MODEL_API_KEY`, `MODEL_A`, `MODEL_B`, `PROMPT_A_VERSION`, `PROMPT_B_VERSION`, `CATALOG_VERSION`, `RULES_PROFILE`, `MAX_JOB_REQUESTS`, `MAX_PROVIDER_CONCURRENCY`, `ALLOWED_CLIENT_VERSION` et les budgets doivent avoir une valeur documentée. Les secrets n'ont pas de valeur par défaut publique. Une clé manquante fait échouer le contrôle de disponibilité de la génération avec un message lisible.

## Sauvegarde et restauration

Sauvegarder la base et les fichiers référencés selon un même point de cohérence logique. Conserver un manifeste des hashes et versions. Un test de restauration reconstruit un environnement vierge, retrouve les captures, les sorts publiés et les jobs en attente, puis lance un sort restauré dans Unity.

Une suppression de support ne doit pas supprimer un fichier encore partagé par un autre enregistrement. Le garbage collector de stockage ne traite que les objets sans référence après une période de grâce. La remise finale inclut un rapport de restauration, pas seulement une commande `backup` non essayée.

## Procédure d'incident

Identifier le job par son `trace_id`, vérifier sa dernière étape durable, distinguer fournisseur indisponible, quota épuisé, erreur de schéma et incompatibilité du catalogue. Reprendre uniquement l'étape autorisée avec un nouvel `attempt_id`. Si le prompt ou le catalogue change, enregistrer cette modification ; ne pas réécrire l'historique du support. Pour un paquet déjà publié, corriger le runtime ou effectuer une migration tracée, pas une nouvelle création silencieuse.

## CHAPITRE 20 / SP-1.0

# Performance et observabilité

Les chiffres ci-dessous sont des **objectifs d'acceptation proposés**, non des mesures réalisées. La machine de référence doit être choisie et enregistrée par l'équipe avec CPU, GPU, RAM, pilote, résolution et build. Sans cette fiche, « 60 FPS » n'est pas une preuve exploitable.

| Domaine             | Objectif proposé                                                      | Méthode                                                                            |
|---------------------|-----------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Atelier de dessin   | Retour visuel p95 inférieur à 50 ms.                                  | Mesure entre réception de l'entrée et affichage sur la machine de référence.       |
| Capture normale     | Interface réactive ; aucun blocage continu supérieur à 100 ms.        | Profilage clôture, encodage, hash et écriture.                                     |
| Scène d'épreuve     | 60 images/s à 1080p ; frame p95 ≤ 16,7 ms et p99 ≤ 25 ms.             | Build autonome, scénario chargé fixé, sans profiler attaché pour la mesure finale. |
| Cœur des sorts      | Coût CPU p95 ≤ 4 ms/frame dans le scénario de charge.                 | Profiler séparant logique, requêtes physiques et rendu.                            |
| Allocations         | Pas d'allocation managée par tick dans les chemins chauds stabilisés. | Mesures après préchauffage ; sérialisation et écrans hors tick séparés.            |
| Endurance           | 30 minutes sans croissance mémoire continue ni instances orphelines.  | Boucle de créations locales de casts depuis paquets déjà compilés.                 |
| Génération distante | Latences A, B, file et total mesurées séparément.                     | Aucun engagement arbitraire avant essais réels.                                    |

Le scénario de charge associe huit casts compatibles avec l'admission globale, 32 acteurs et 32 objets physiques ; les limites exactes de la scène sont enregistrées dans un preset de test technique. Ce preset ne représente pas un sort préconstruit disponible au joueur.

Tracer `capture_time`, `queue_wait`, `model_a_latency`, `geometry_time`, `model_b_latency`, `compile_time`, `publish_time`, nombre de réparations et usage fournisseur. Côté Unity : casts actifs, instances, requêtes physiques, applications d'effets, particules, temps CPU et GPU. Une lenteur d'API n'est jamais convertie en puissance, mana ou cooldown.

La dégradation visuelle éventuelle diminue particules secondaires, distorsions et lumières accessoires. Elle ne modifie ni trajectoire, ni masque d'effet, ni dégâts, ni nombre de cibles. Les paramètres de qualité sont disponibles dans le laboratoire et testés sur les mêmes paquets.

## CHAPITRE 21 / SP-1.0

# Plan de tests et validation humaine

## Quatre couches de preuve

**Contrats** : schémas, versions, exemples, références, bornes et formats d'API. Ils ne prouvent ni la qualité du modèle ni le fonctionnement d'un RigidBody. **Cœur** : états, compilation, graphe, unités, effets et ordonnancement dans un monde de test déterministe. **Unity** : scène réelle, collisions, rendu, audio, sauvegarde et build autonome. **Humains** : correspondance au dessin, cohérence, variété, lisibilité et plaisir de l'effet.

L'agent peut exécuter les trois premières couches et produire des preuves. Le verdict esthétique ou de gameplay est inscrit par l'équipe. Le logiciel final n'exige pas une modération humaine pour chaque nouveau parchemin ; cette validation porte sur le développement, les corpus et la mise en service.

## Matrice minimale des essais automatisés

| Famille         | Cas obligatoires                                                                                                                                                     |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dessin          | Clic simple ; pointillé à 30/60/144 Hz ; double trace gribouillée ; région verrouillée ; geste traversant deux régions ; épuisement en plein geste ; perte de focus. |
| Engagement      | Fermeture vierge ; fermeture après un point ; trois régions terminées ; crash après journal durable ; reprise sans édition ; deux transmissions identiques.          |
| Interprétation  | Deux vraies images envoyées ; aucune propriété de rôle venant du brush_id ; sortie JSON ; refus ; réponse tronquée ; texte hostile dans l'image.                     |
| Traduction      | Clause manquante ; soin ajouté ; filtre changé ; géométrie vide ; effet non implémenté ; cycle ; parent inexistant ; nombre hors bornes.                             |
| Projectile      | Mur mince à vitesse maximale ; contact initial ; rebond ; pénétration ; cible à plusieurs colliders ; guidage sans cible ; expiration unique.                        |
| Autres porteurs | Chaîne sans revisite ; faisceau bloqué ; trou dans une zone ; onde balayant une cible entre deux ticks ; piège avec cible déjà présente ; barrière détruite.         |
| Effets          | Santé bornée ; pas de résurrection ; masse et impulsion ; burn/wet dans les deux ordres ; slow non cumulatif ; filtres allié/soi/environnement.                      |
| Reprise serveur | Worker tué après A ; lease expiré ; ancienne réponse tardive ; publication concurrente ; base temporairement indisponible ; quota épuisé.                            |

| Famille      | Cas obligatoires                                                                                                               |
|--------------|--------------------------------------------------------------------------------------------------------------------------------|
| Paquet       | Corruption d'octets ; version inconnue ; géométrie manquante ; comparaison noyau/Unity ; chargement offline ; profil ancien.   |
| Présentation | Frontière visible/physique ; qualité basse ; arrêt des sons ; retour au pool ; changement de scène ; résolution et UI clavier. |

Ajouter des tests génératifs qui fabriquent des plans **valides au catalogue** pour exercer les limites, ainsi que des mutations invalides. Ils utilisent des données de test explicitement identifiées. Ils ne doivent pas être présentés comme mille sorts inventés et jugés intéressants par l'IA.

## Corpus humain proposé

Préparer 30 dessins de conception pour affiner la grammaire et les prompts, puis 30 dessins inédits conservés hors réglage pour la recette finale. Parmi ces derniers : dix paires créées par deux personnes à partir d'intentions proches, cinq dessins maladroits/partiels et cinq compositions multi-étapes. Les auteurs conservent leur intention initiale séparément de la sortie du modèle.

L'équipe observe le dessin, la description de A, le plan et une vidéo du cast. Elle note : fidélité à l'image, cohérence des règles, signature visuelle, écart par rapport à l'intention de l'auteur, lisibilité des effets et anomalies. Un sort peut être différent de l'intention mais cohérent avec un dessin raté : les deux notions sont distinguées.

Seuils proposés pour SP-1.0 : aucune capacité promise mais absente dans les 30 cas ; au moins 24 cas sur 30 jugés cohérents avec l'image par les deux créateurs ; aucun des dix couples ne doit être distinguable uniquement par le titre ou l'identifiant. Ces seuils sont des critères de recette à accepter, pas une estimation statistique de la qualité future sur toutes les images.

## Contenu d'une revue

Identité du cas, date, personne, commit, version Unity, prompts, modèles, catalogue, hashes de capture/description/plan/paquet, vidéo ou captures, résultats techniques et verdict humain. Les statuts proposés sont `accepted`, `changes_requested` ou `rejected`, avec commentaire obligatoire. La revue ne modifie pas le parchemin : elle crée une preuve et éventuellement une tâche de correction du système.

La fiche livrée dans `contracts/human-review.schema.json` exige une identité de relecteur et une attestation de soumission humaine. Ce champ n'est pas une preuve cryptographique de présence humaine ; le contrôle réel est l'interface authentifiée et la procédure de revue. Un agent n'est pas autorisé à le remplir pour conclure à la qualité de son propre travail.

## CHAPITRE 22 / SP-1.0

# Ordre de réalisation et jalons

## M0 — Dépôt et contrats

Créer le dépôt, verrouiller les outils, intégrer les schémas, établir le catalogue et le profil, créer les assemblages partagés et les tests de validation. Livrer un build Unity vide qui se lance, ainsi qu'un serveur répondant à son healthcheck. Fournir les commandes exactes et une preuve sur la machine de build. **Porte humaine** : l'équipe vérifie le périmètre et les arbitrages P01-P10 ; aucun travail de monde ou de personnage ne commence.

## M1 — Parchemin final, sans faux sort

Réaliser l'atelier, les traces, le budget, les régions, la clôture, le journal et les captures. Le modèle n'est pas encore nécessaire pour les tests de raster, mais l'interface ne prétend pas générer un sort. Livrer PNG, journal et cas de crash. **Porte humaine** : vous dessinez et jugez si l'engagement au lever de pinceau est utilisable. Une modification de cette règle est versionnée ici avant la suite.

## M2 — Lecture multimodale réelle

Brancher A, archiver les entrées et sorties, montrer la description et ses observations, gérer les erreurs réelles. Ne pas auto-évaluer si le sort est « bon ». **Porte humaine** : corpus de conception, vrais dessins, accord sur la manière dont les régions et traces sont lues.

## M3 — Description, plan et compilation

Brancher B, produire des plans, valider toutes les clauses et géométries, calculer les limites et construire les paquets. Les fixtures permettent de tester le compilateur avant de dépenser des appels, mais la démonstration du jalon utilise des descriptions réelles issues de M2. **Porte humaine** : comparaison de la description et des règles, sans effet ajouté ou retiré.

## M4 — Tous les porteurs et tous les effets

Implémenter le runtime, les six porteurs, les six effets, les réactions, les événements composés, les collisions et les récepteurs. Mettre à disposition la scène de mesure. Des renderers simples sont permis pendant le travail, pas comme remise finale. **Porte humaine** : essais réels, paramètres visibles et comportements correspondant aux clauses.

## M5 — Géométrie et présentation finales

Finaliser extraction, trajectoires, masques, barrières, matériaux, particules, sons et interface. Régler la signature du parchemin. **Porte humaine** : dessins reconnaissables dans les sorts, revue des dix paires, absence de VFX générique universel.

## M6 — Robustesse et exploitation

Finaliser authentification, reprise locale/serveur, idempotence, plafonds de coût, monitoring, sauvegarde/restauration et chargement hors ligne. Tester les coupures aux étapes importantes.

**Porte humaine** : démonstration d'une panne réelle simulée puis reprise sans nouvelle édition ni second sort publié.

## M7 — Livraison et recette

Construire le client Windows autonome, déployer le service de test, exécuter les 30 cas inédits et le scénario de charge. Livrer code, assets, configuration d'exemple, installation, preuves techniques, revues humaines et défauts résiduels. **Porte finale** : vous acceptez la boucle complète ; l'agent ne transforme pas « tests verts » en approbation de gameplay.

Les jalons sont un ordre de dépendance, pas une autorisation de s'arrêter après M2. La cible demandée est M7. Pendant l'implémentation, un agent peut continuer les travaux techniques indépendants, mais il ne peut pas inventer l'accord d'une porte humaine ou modifier les intentions pour la contourner.

**CHAPITRE 23 / SP-1.0**

# Définition de terminé et intégration au futur jeu

## Preuve de bout en bout exigée

Sur une installation propre, un créateur reçoit un parchemin vierge, dessine une composition non présente dans les fixtures, termine selon les règles, obtient une description avec un vrai appel de A, obtient un plan avec B, lance le résultat dans la scène et observe ses effets. Il ferme le client, coupe l'accès au fournisseur, relance et utilise le même sort sans changement de paramètres ni appel de génération.

Un second essai provoque une panne pendant B. La tâche reprend depuis la description persistée, sans nouveau dessin, sans nouveau support et sans seconde publication. Un troisième essai modifie les octets d'un paquet local : le client détecte la corruption et ne l'exécute pas. Ces essais doivent être enregistrés dans les preuves finales.

## Checklist de livraison logicielle

Le dépôt contient tous les fichiers de scène, paramètres de projet, assets et dépendances nécessaires ; pas seulement des scripts à coller dans un projet à la main. Le build autonome démarre sans mode développeur. Le service s'installe avec une configuration documentée. Les secrets ne sont pas présents dans les sources, l'exécutable ou les exports de diagnostic.

Chaque capacité exposée au modèle a des tests, un comportement, un rendu et un son fonctionnels. La fiche d'un sort correspond à son paquet. Toutes les erreurs connues sont listées honnêtement. Aucun bouton visible ne pointe vers une fonction vide. Les données simulées sont confinées aux tests ; elles ne se substituent pas au chemin nominal.

## Surface d'intégration à conserver

Le futur jeu devra fournir l'identité du lanceur, les relations allié/ennemi, les récepteurs de santé/statut/impulsion, le point visé, les ressources de lancement et le stockage de bibliothèque. Le moteur retourne un `CastResult` et des événements, sans connaître un système de quête, des classes ou une économie.

SP-1.0 n'instaure pas de mana et de cooldown définitifs. Le laboratoire utilise seulement un anti-spam technique de 250 ms entre demandes de cast du même lanceur et l'admission globale. Cette temporisation n'est pas une caractéristique permanente du sort et ne dépend jamais du temps du modèle. Les coûts de gameplay pourront être intégrés via `ICastPolicy` sans changer le dessin, la description ou la physique des porteurs.

Une future autorité serveur de combat pourra fournir un autre `ISpellWorld` et une autre politique d'admission. Cette séparation prépare une intégration ; elle n'implémente ni réseau multijoueur ni prédiction client dans cette livraison.

## CHAPITRE 24 / SP-1.0

# Consigne à donner à l'agent de développement

Le texte complet prêt à transmettre est dans `prompts/00_AGENT_BUILD.md`. La consigne fondamentale est : **réaliser le produit du présent périmètre, pas une preuve de concept qui en imite la surface**. Les documents et schémas sont l'autorité de contrat ; les noms de classes proposés peuvent évoluer si les responsabilités et les tests sont conservés et si le changement est documenté.

L'agent commence par lire les décisions acquises et les exclusions, puis les schémas et la matrice de recette. Il produit un état initial vrai de son environnement : Unity disponible ou non, version réelle, capacité de compiler Windows, accès fournisseur, état des tests. Il ne prétend pas avoir utilisé Unity, appelé un modèle ou visionné une vidéo si cela n'a pas eu lieu.

Il livre à chaque jalon une version lançable, des preuves et une liste de défauts. Il ne crée pas de note « approuvé par les créateurs » sans leurs actions. Il ne substitue pas un algorithme de détection de symboles au modèle multimodal, n'ajoute pas de gomme et ne change pas l'objectif en construction d'un RPG complet.

Le dossier de spécification livré ici contient des fixtures **illustratives** et un validateur de contrats. Son rapport ne vaut pas validation du jeu à construire. Les appels API réels, les tests Unity, les performances et les revues humaines restent des preuves que le développement devra produire.

## CHAPITRE 25 / SP-1.0

# Sources et traçabilité

## Sources de conception

**[T1]** `Conception_de_sorts_par_parchemins_dessin_s_2026-09-19.txt`, discussion du 19 septembre 2026, 15:01:48–15:30:47. Sert notamment à la distinction entre propriétés composées et liste de sorts, au rôle spatial du layout et à la réutilisation du parchemin. Les propositions contredites dans T2 ne sont pas conservées comme décisions.

**[T2]** `AI_Interpreted_Spell_Drawing_System_2026-09-19.txt`, discussion du 19 septembre 2026, 16:30:42–17:07:29. Les passages déterminants sont référencés au chapitre 02. Le périmètre « seulement la description » est désormais remplacé par votre demande actuelle de boucle complète sous Unity.

**[T3]** Demande actuelle : « moteur dessin to description et description to spell via Unity », « version finale juste cette boucle ». Source du choix de moteur et du niveau de finition attendu.

Les arbitrages P01–P10, les contrats, les nombres, la palette, le catalogue de SP-1.0, les tests et l'architecture sont les propositions de ce dossier. Ils ne doivent pas être attribués rétroactivement à vos conversations.

## Références techniques officielles consultées le 19 septembre 2026

**[W1] Unity — manuel Unity 6.3 LTS et backend IL2CPP.** La page identifie la ligne 6000.3 comme Unity 6.3 LTS et décrit la compilation anticipée. <https://docs.unity3d.com/6000.3/Documentation/Manual/scripting-backends-il2cpp.html>

**[W2] Unity — introduction URP.** Documentation du pipeline retenu ; le choix de l'utiliser est un arbitrage de ce cahier des charges. <https://docs.unity3d.com/6000.3/Documentation/Manual/urp/urp-introduction.html>

**[W3] Unity — profils .NET.** .NET Standard 2.1 et contraintes de compatibilité des bibliothèques. <https://docs.unity3d.com/6000.3/Documentation/Manual/dotnet-profile-support.html>

**[W4] Microsoft — cycle de vie .NET.** Existence et période de support de .NET 10 ; ce document ne suppose pas qu'un assembly .NET 10 soit chargeable dans Unity. <https://learn.microsoft.com/en-us/lifecycle/products/microsoft-net-and-net-core>

**[W5] Unity — Input System, guide de démarrage.** Actions et configuration des entrées. La version exacte du package est à verrouiller dans le projet, pas déduite de cette URL d'exemple. <https://docs.unity3d.com/Packages/com.unity.inputsystem@1.14/manual/QuickStartGuide.html>

**[W6] Unity — IL2CPP.** Compilation AOT et nécessité de tester le build cible. Même source que W1.

**[W7] Unity — AsyncGPUReadback.** API de récupération asynchrone de données GPU. <https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Rendering.AsyncGPUReadback.html>

**[W8] OpenAI — images et vision.** Plusieurs images en entrée et coût des images. <https://developers.openai.com/api/docs/guides/images-vision>

**[W9] OpenAI — sorties structurées.** Schémas stricts et traitement des refus/réponses incomplètes. <https://developers.openai.com/api/docs/guides/structured-outputs>

**[W10] OpenAI — catalogue de modèles.** Source de l'identifiant de modèle proposé ; aucune comparaison sur votre corpus n'a encore été réalisée ici. <https://developers.openai.com/api/docs/models>

**[W11] Unity — Physics.SphereCastNonAlloc.** Volume balayé et précautions sur les résultats bornés. <https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Physics.SphereCastNonAlloc.html>

**[W12] Unity — ObjectPool.** API de pool ; la politique de remise à zéro relève de l'implémentation. [https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Pool.ObjectPool\\_1.html](https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Pool.ObjectPool_1.html)

**[W13] PostgreSQL 17 — SELECT et clauses de verrouillage.** Base de la sélection de jobs avec `SKIP LOCKED` ; le protocole de lease est notre proposition. <https://www.postgresql.org/docs/17/sql-select.html>

**[W14] OpenAI — référence générale de l'API.** Gestion des clés côté serveur. <https://developers.openai.com/api/reference/overview>

ANNEXE A / SP-1.0

# Règles des contrats

Cette annexe complète le cahier principal. Les JSON Schema décrivent la forme des données ; les règles ci-dessous décrivent les contraintes entre champs. Les exemples sont des fixtures manuelles et ne prouvent aucune interprétation ou exécution Unity.

## A • Autorité des fichiers

| Fichier                                                     | Producteur prévu                           | Consommateur                                   |
|-------------------------------------------------------------|--------------------------------------------|------------------------------------------------|
| drawing-capture.schema.json                                 | Client, puis vérification serveur          | API de capture.                                |
| spell-description.schema.json                               | Modèle A                                   | Géométrie, modèle B et revue.                  |
| spell-plan.schema.json                                      | Modèle B                                   | Compilateur.                                   |
| geometry.schema.json                                        | Résolveur géométrique contrôlé             | Compilateur et Unity.                          |
| compiled-spell.schema.json                                  | Compilateur contrôlé                       | Unity et fiche de règles.                      |
| human-review.schema.json                                    | Interface de revue avec une action humaine | Historique de recette.                         |
| capability-catalog.json                                     | Développeurs                               | A, B, validateur et runtime.                   |
| model-a.response-format.json / model-b.response-format.json | Configuration contrôlée                    | Format de réponse du fournisseur de référence. |
| openapi.yaml                                                | Développeurs                               | Client, API et tests de service.               |

La forme de contrat prime sur une abréviation dans la prose. Une contradiction découverte entre schéma, catalogue et texte bloque une release : elle doit être corrigée dans les trois, pas arbitrée silencieusement par l’agent.

## B • Description et traçabilité

Les identifiants d’observations et de clauses sont uniques. Toutes les références d’observation existent. Chaque sujet mécanique possède exactement un porteur principal dans le plan. Un sujet peut recevoir plusieurs effets, décrits par plusieurs faits `effect`. Un fait ne peut pas être satisfait simplement par l’existence d’un effet ailleurs dans le sort.

`relations` décrit les liens entre sujets : source, événement, cible et maximum d’activations. C’est la version contrôlable de « au premier contact, créer une zone ». Une racine n’a pas de relation entrante. SP-1.0 autorise au plus une relation entrante par sujet ; les fusions multi-parents sont exclues. B reproduit chaque relation dans `activation` sans en créer d’autre. Les sujets sans parent deviennent des racines.

La relation cite la clause qui la justifie. Un fait de `motion` vaut `stationary` pour une zone, une barrière ou un piège, `expanding` pour une onde, et une option explicite pour un projectile. Pour le faisceau, la notion de mouvement n'est pas nécessaire : la clause décrit ses événements et sa géométrie.

Un sujet géométrique doit avoir un `shape_request`. Son porteur principal utilise une géométrie appartenant à la région et au rôle demandés. Sa silhouette supplémentaire peut utiliser le dessin entier. Une clause purement visuelle n'autorise pas une nouvelle application d'effet.

## C · Événements disponibles et application des effets

Un porteur peut émettre plus d'événements qu'il ne possède de récepteurs d'effet immédiats. `spawn` et `expire` ne ciblent pas arbitrairement un personnage voisin. Le tableau suivant est contractuel pour `effects[].event` :

| Porteur    | Événements autorisés pour un effet direct                                                |
|------------|------------------------------------------------------------------------------------------|
| Projectile | <code>hit</code>                                                                         |
| Faisceau   | <code>hit</code> ; les hits sont produits à chaque cadence pour un faisceau entretenu.   |
| Champ      | <code>enter</code> , <code>tick</code>                                                   |
| Onde       | <code>hit</code>                                                                         |
| Barrière   | <code>block</code> , uniquement si l'événement possède un récepteur matériel admissible. |
| Piège      | <code>trigger</code>                                                                     |

Les enfants peuvent être déclenchés par les événements exposés dans `capability-catalog.json`. `spawn` n'est pas admis comme événement de relation dans cette version : placer plusieurs racines ou un délai de racine. Le modèle ne doit pas inventer une cible pour appliquer un dégât à `expire`. Il peut déclencher une onde sur `expire`, qui effectuera sa vraie recherche de cibles.

Un projectile détruit contre un mur émet `expire` mais pas `hit` sur un acteur inexistant. `hit` signifie ici rencontre d'un récepteur ; le filtre de ses effets détermine ensuite ce qui s'applique. Pour un champ enfant devant apparaître également contre un mur, la description doit prévoir la relation d'expiration appropriée dans une version permettant les deux conditions ou accepter que ce cas reste hors de cette recette. Ne pas prétendre que `hit` et `expire` sont interchangeables.

Dans le cas illustratif, le champ naît sur le premier `hit` admissible du projectile. Son `contact_filter` vaut `hostile` : toucher un allié ou un mur n'émet donc pas cet événement. Le projectile peut y être arrêté matériellement et émettre `expire`, mais il n'y crée pas ce champ. Cette exclusion figure dans la fiche de règles.

## D · Acquisition et filtres de porteurs

Pour un projectile, `contact_filter` définit les récepteurs qui émettent `hit` et, en mode guidé, les cibles recherchées. Il vaut aussi pour les projectiles droits et courbes. Un récepteur exclu reste un obstacle si ses couches physiques le rendent solide ; il arrête alors le projectile et émet seulement `expire`. Les dégâts à une barrière restent traités par son adaptateur de collision.

Pour un faisceau, `chain_filter` définit le filtre des récepteurs visés, y compris le premier. Pour un piège, `trigger_filter` définit l'entrée admissible. Pour champ/onde, l'union des filtres d'effets et des besoins des enfants définit les récepteurs candidats. Lorsqu'un champ sans effet déclenche un enfant sur `enter`, ce déclenchement considère `all_actors` par convention de SP-1.0 ; cette règle doit être exposée dans la fiche finale. Une future garde par relation demanderait un nouveau champ de contrat.

Ces règles sont une raison de conserver des faits de ciblage dans A et de les comparer à B. Les filtres des effets ne doivent pas dépasser l'intention du sujet. Un champ peut combiner des effets ciblant différemment à condition que chaque filtre figure dans les faits de ce sujet.

## E · Contraintes par porteur

**Racines :** `parent_id = null`, `event = cast`, `max_activations = 1`; ancrage `caster` ou `aim_point`.

**Enfants :** parent existant, événement effectivement émis par lui, ancrage `parent_event`. Le modèle ne produit pas un enfant à l'origine du monde par défaut.

**Copies :** autorisées de 1 à 8. L'éventail répartit les angles uniformément entre `-spread/2` et `+spread/2` ; avec une copie, angle zéro et spread zéro. Les porteurs persistants superposés ne gagnent aucun effet implicite : les statuts suivent leur règle de cumul, les dégâts directs explicitement multiples s'appliquent. Une composition inutilement coûteuse peut échouer au budget.

**Projectile :** `turn_mdeg_s = 0` hors guidage ; une courbe exige une géométrie `path`. Vitesse et portée sont positives. Après le dernier point d'un chemin courbe, poursuivre sur sa tangente finale ; ne pas boucler. Le guidage est un mode de trajectoire séparé, qui ne garantit pas de suivre l'intégralité du chemin dessiné. Le premier terme atteint entre portée, durée et arrêt matériel termine l'instance. Le budget de `hit` est borné par `1 + pierces + bounces`, avec déduplication par récepteur.

**Faisceau :** si `lifetime_ticks = 1`, l'évaluation est instantanée. Sinon, `tick_interval ≥ 5`. `chain_radius_cm = 0` lorsque `chain_hops = 0` ; rayon positif dans le cas inverse. Premier hit au tick de création, puis toutes les `tick_interval` unités jusqu'à expiration exclusive. La géométrie est un chemin de silhouette ; chaque segment effectif entre cibles conserve ses collisions propres.

**Champ :** première cadence à `spawn + tick_interval` ; `enter` une fois par cible pour la vie de l'instance. À l'expiration, les effets sont traités avant retrait seulement lorsque le tick de cadence tombe exactement sur la limite. Maximum 32 récepteurs distincts sur la vie ; les suivants sont ignorés selon l'ordre déterministe, limite affichée.

**Onde :** sa durée est la durée d'expansion. Rayon contractuel du front en fonction du tick : `radius_cm × elapsed / lifetime_ticks`. Le masque est testé dans son repère dimensionné par `scale_cm`, puis intersecté avec la bande du front. Pour une empreinte non centrée, le front part de l'origine locale ; le rayon doit couvrir la portée utile voulue. La valeur `front_width_cm` ne dépasse pas `radius_cm`.

**Barrière :** géométrie `path`, épaisseur et hauteur positives ; structure positive ; expiration au premier terme durée, structure nulle ou maximum de blocages. Le nombre de segments utilisé n'excède pas 64. Le contrôle d'orientation et de placement refuse une création imbriquée dans le lanceur ou un autre volume bloquant ; c'est un refus de cast, pas un changement du parchemin.

**Piège :** `arm_ticks < lifetime_ticks`, `rearm_ticks ≥ 1` ; compteur de déclenchements global par instance. Une cible déjà présente à l'armement déclenche si le filtre le permet. Après le maximum, l'instance expire sans second événement parasite.

## F • Contraintes des effets

`damage`, `heal`, `impulse` ont `duration_ticks = 0`. Les statuts ont une durée strictement positive. `burn` utilise un multiple de 50 ticks. `wet.amount = 0`. `slow.amount ≤ 750`. `impulse.direction` est autre que `none` ; tous les autres effets ont `direction = none`.

Les plafonds `amount` sont ceux du catalogue : dégâts/soin 100 000 milli-points, impulsion 200 000 milli-N-s, brûlure 10 000 milli-points par seconde, Mouillé zéro, ralentissement 750 millièmes. Ce sont des bornes de laboratoire, pas des valeurs équilibrées pour un RPG.

Une impulsion vers l'extérieur utilise la direction normalisée de la source de l'événement au centre du récepteur. Si les positions coïncident, prendre la direction avant du cast ; jamais diviser par zéro. Pour un effet intérieur, inverser cette direction. `up` est l'axe vertical. Les acteurs sans RigidBody implémentent l'impulsion dans leur adaptateur, sans modifier arbitrairement leur transform par la couche VFX.

## G • Géométrie et fichiers référencés

Les points de géométrie sont stockés sur deux axes en entiers de -10 000 à 10 000. La dimension maximale de l'asset correspond à `scale_cm`, donc un point normalisé `x` devient `x × scale_cm / 20 000` centimètres, avant rotation et translation. Les coordonnées de chemin de test ne sont pas des coordonnées de pointeur et ne sont pas envoyées à A à la place du dessin.

`mask_file` dans le contrat de géométrie désigne uniquement un nom de fichier relatif produit par le résolveur dans un paquet d'asset contrôlé. Il ne vient pas du modèle B. Interdire séparateurs, chemin absolu et `..` ; la liste des fichiers du paquet est manifeste et hashée. À l'API, les fichiers sont résolus par identifiants d'artefacts et soumis à l'autorisation, pas servis via le nom brut.

Le paquet compilé contient `geometry_manifest` pour les JSON de géométrie et `binary_assets` pour leurs PNG. Chaque `mask_file` correspond exactement à un `binary_assets.file_name` unique ; son `artifact_id` permet le téléchargement autorisé et son SHA-256 vérifie les octets. Le masque n'est pas implicitement fiable parce que le JSON qui le référence est hashé.

`provenance` conserve le mode, les empreintes de capture/référence et les versions de modèles et prompts. Dans les fixtures manuelles, les modèles et identifiants de réponse restent `null`, avec `mode = fixture` ; ils ne doivent pas imiter des identifiants de vraies requêtes. En production, les données viennent des réponses archivées et non d'un texte inventé par B.

Une empreinte exige un masque existant et borné. Un chemin exige au moins deux points distincts et une longueur non nulle. Une silhouette peut référencer un masque sans collision. La banque doit compter également les fichiers de masque dans le budget mémoire, pas seulement le JSON.

## H · Bornes et extinction complète

La borne temporelle du cast inclut le dernier statut persistant appliqué, même s'il survit au champ qui l'a créé. Pour le cas de référence, une borne conservatrice est 125 ticks de projectile + 1 tick de passage à l'enfant + 200 ticks de champ + 150 ticks de brûlure, soit **476 ticks**. Une borne à 326 oublierait le statut résiduel.

Le calcul du nombre d'effets compte chaque application de statut ainsi que son nombre maximal de ticks. Il peut surestimer, puisque les statuts se rafraîchissent au lieu de s'empiler, mais jamais sous-estimer. Une fois un statut transféré au `StatusSystem`, celui-ci conserve source et identité du cast sans maintenir un `GameObject` de projectile disparu.

Le script QA livré vérifie des formes et quelques invariants. Il ne remplace pas ce calcul pessimiste complet ni les tests du runtime à développer.

## ANNEXE B / SP-1.0

# Branchement réel du fournisseur

Ce document précise le branchement à réaliser dans le backend. Aucun appel réel n'est exécuté par le dossier livré. Les exemples de corps de requête sont des instructions de construction, pas des clés ou configurations déjà actives.

## 1 • Une clé côté serveur, aucun secret dans Unity

Le backend lit `OPENAI_API_KEY` depuis son gestionnaire de secrets ou son environnement. Unity appelle uniquement l'API du laboratoire avec son jeton utilisateur distinct. Le fournisseur de référence est l'API Responses d'OpenAI ; son endpoint documenté est <https://api.openai.com/v1/responses>. La documentation officielle est référencée aux sources W8, W9, W10 et W14 du cahier.

L'intégration implémente `IMultimodalInterpreter` pour A et `IDescriptionPlanner` pour B. Leur contrat de résultat distingue réponse complète, refus, contenu incomplet, erreur transitoire et erreur permanente. Le SDK éventuel et sa version sont verrouillés dans le dépôt après le test de branchement. Ne pas assimiler un HTTP 200 à un document métier valide.

## 2 • Corps de requête A

Construire le corps JSON suivant avec un sérialiseur ; ne pas effectuer de remplacement de chaînes à partir du dessin. Les mentions entre chevrons ci-dessous désignent des valeurs construites par le serveur.

```
{
  "model": "<MODEL_A>",
  "store": false,
  "input": [
    {
      "role": "system",
      "content": [{"type": "input_text", "text": "<01_MODEL_A_INTERPRETE.md + layout + catalogue>"}]
    },
    {
      "role": "user",
      "content": [
        {"type": "input_text", "text": "Image 1 : référence du layout. Image 2 : dessin réel à interpréter."},
        {"type": "input_image", "image_url": "data:image/png;base64,<REFERENCE_PNG>"},
        {"type": "input_image", "image_url": "data:image/png;base64,<DRAWING_PNG>"}
      ]
    }
  ],
  "text": {"format": "<OBJET_JSON_model-a.response-format.json>"}
}
```

Dans le corps réel, `text.format` est **l'objet JSON chargé du fichier**, pas la chaîne montrée pour la lisibilité de cet exemple. Les images sont de vraies données PNG validées. Aucun identifiant de pinceau ne devient une consigne de dégâts ou de soin. Les guides régionaux restent dans le prompt et dans l'image de référence.

Le serveur calcule les hashes des images et du prompt avant l'appel. Il conserve les octets exacts de la première description acceptée techniquement. Les réparations structurelles de A restent limitées aux erreurs ; elles ne proposent pas plusieurs sorts au joueur.

### 3 • Corps de requête B

Employer la même structure avec `MODEL_B` et `model-b.response-format.json`, mais sans `input_image`. Le message système contient `02_MODEL_B_TRADUCTEUR.md`. Le message utilisateur contient un objet sérialisé composé de :

```
source_description_sha256 : hash exact du document de A archivé
source_description       : document SpellDescription complet
authorized_catalog       : catalogue réellement compilable dans ce build
geometry_bank            : IDs, régions, rôles, mesures, propriétés disponibles
rules_profile            : bornes lab_v1 et unités
```

La banque contient les résultats du résolveur, pas un chemin de fichier libre à ouvrir par le modèle. Les masques eux-mêmes sont gérés par le compilateur et le runtime ; B peut choisir des références autorisées. B recopie l'empreinte fournie ; **il ne calcule pas un SHA-256 mentalement**.

L'étape B n'emploie pas de fonction permettant d'exécuter du code ou d'installer une mécanique. La sortie est validée selon le schéma, puis selon les clauses, relations, géométries et budgets. Une réponse non représentable est un incident technique à résoudre, jamais une publication avec des champs ignorés.

### 4 • Préflight obligatoire avant une série de générations

Vérifier successivement l'accès aux identifiants de modèles, un appel à deux images, une réponse A conforme, une réponse B conforme avec références locales de schéma, et la gestion d'un refus ou résultat incomplet. L'utilisation d'un schéma JSON localement correct ne prouve pas que le fournisseur accepte tous ses mots-clés dans ce mode de réponse. Corriger l'enveloppe de transport en conservant le contrat métier, puis enregistrer ce préflight dans le dépôt.

Ne pas injecter automatiquement des paramètres `temperature`, `seed`, `reasoning` ou des niveaux supposés compatibles. Leur disponibilité dépend du modèle et du mode choisis. Enregistrer uniquement ceux réellement pris en charge et testés. Le modèle recommandé est un candidat initial, pas une assurance d'accès avec le compte de l'équipe.

### 5 • Lecture et gestion du résultat

Parcourir les éléments de sortie typés plutôt que supposer que le premier élément est toujours du texte. Une sortie de refus ou de contenu incomplet ne passe pas dans le parseur de `SpellDescription`. Une sortie textuelle JSON est lue avec taille maximale, rejet des clés dupliquées, profondeur bornée et contrôle des valeurs finies.

Conserver le modèle retourné, l'identifiant de requête quand disponible, les usages facturables, les dates, la version de prompt et le hash de la tentative. N'inscrire aucune clé d'API dans les logs. Le client reçoit l'étape de travail et un identifiant de diagnostic ; les détails du fournisseur ne sont pas affichés comme une propriété du sort.

Le timeout de 120 secondes est une limite de travail proposée, non une promesse de latence. Une limite de sortie doit être dimensionnée après mesure des schémas ; une troncature est un incident distinct de l'indéchiffrabilité d'un dessin. Les compteurs de tentatives sont persistants et respectent le budget global du cahier.

## 6 • Mode concepteur isolé

`POST /v1/authoring/plan` accepte une `SpellDescription` structurée rédigée ou modifiée par un humain, accompagnée de références de géométries accessibles. Il n'appelle pas A. Il produit une tâche de diagnostic qui parcourt B, le compilateur et Unity.

Les géométries utilisées sont sélectionnées dans un dossier de diagnostic importé par l'outil d'administration ou dans une capture existante autorisée. Ne pas accepter de chemin absolu saisi dans un formulaire. Ce chemin ne crée pas un bouton « modifier » sur un parchemin joueur et ne convertit pas automatiquement une phrase libre en tous les champs structurés.

## ANNEXE C / SP-1.0

# Les 30 tickets de construction

**État initial de tous les travaux : à réaliser.** Les schémas du dossier sont des spécifications ; ils ne signifient pas qu'un module ci-dessous existe déjà. Les portes humaines sont des points de revue de l'équipe, pas des autorisations de considérer un sous-ensemble comme version finale.

| Ticket | Lot | Réalisation attendue                                                              | Preuve de sortie                                                 |
|--------|-----|-----------------------------------------------------------------------------------|------------------------------------------------------------------|
| B01    | M0  | Créer dépôt Unity 6.3 LTS, API et bibliothèques partagées ; verrouiller versions. | Build Windows IL2CPP vierge, versions et scripts de build.       |
| B02    | M0  | Générer/écrire DTO C# compatibles Unity depuis les contrats.                      | Aller-retour JSON sans perte sur Editor et build.                |
| B03    | M0  | Portes de qualité : corpus privé, dossiers de preuve et action de revue humaine.  | Cas et reviewers enregistrés ; aucune approbation automatique.   |
| B04    | M1  | Définir layout, transformations pointeur, zoom, palettes et trois traces.         | Traces comparables à différentes fréquences d'images.            |
| B05    | M1  | Raster canonique, masque d'encre et journal durable.                              | Empreintes reproductibles et rejeu du journal.                   |
| B06    | M1  | Encre, verrouillage par région et finalisation sans confirmation.                 | Tests limites de trait, régions traversées, fermeture partielle. |
| B07    | M1  | Perte de focus, arrêt brutal, reprise locale et budget d'encre.                   | Incident enregistré ; pas de retour à un support vierge.         |
| B08    | M2  | API de capture, authentification privée, stockage et clés d'idempotence.          | Double upload sans double tâche ni remplacement du dessin.       |
| B09    | M2  | Worker durable, leases, heartbeats, fencing et reprise des tâches.                | Redémarrage forcé sans double publication.                       |
| B10    | M2  | Connecter A à deux vraies images ; gérer toutes les issues fournisseur.           | Réponses brutes, usages et descriptions réelles archivées.       |
| B11    | M2  | Afficher interprétation intermédiaire et vrais états de travail.                  | Interface sans faux pourcentage ni choix de reroll.              |

| Ticket | Lot | Réalisation attendue                                                           | Preuve de sortie                                                             |
|--------|-----|--------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| B12    | M3  | Implémenter masques régionaux, chemins, empreintes et silhouettes.             | Tests trous, composantes, traits ouverts/fermés et cas dégénérés.            |
| B13    | M3  | Connecter B à la description figée et à la banque géométrique.                 | Descriptions indépendantes de A traduites en plans valides.                  |
| B14    | M3  | Implémenter traçabilité des clauses, sujets, relations et géométries.          | Tests où B ajoute soin, change cible ou perd un enfant, tous bloqués.        |
| B15    | M3  | Compilateur de budgets et graphe acyclique ; génération de la fiche factuelle. | Borne pessimiste incluant les statuts résiduels et tests négatifs.           |
| B16    | M3  | Paquet immuable, manifestes JSON/PNG, cache et contrôle de versions.           | Un octet modifié est détecté ; aucun chargement partiel.                     |
| B17    | M4  | Horloge, scheduler d'événements, admission des casts et pooling.               | Arrêts, reprises, saturation et annulation sans objets orphelins.            |
| B18    | M4  | Projectile droit/courbe/guidé, collisions balayées, rebond et pénétration.     | Mur fin, cibles multiples, double collider et filtre de contact.             |
| B19    | M4  | Faisceau entretenu, occlusion et chaîne sans revisite.                         | Ordre stable et cadence mesurée dans le build.                               |
| B20    | M4  | Champ et onde avec empreintes exactes.                                         | Une cible dans un trou ne reçoit rien ; front rapide ne saute pas une cible. |
| B21    | M4  | Barrière solide et piège armé/réarmé.                                          | Structure, blocages, expiration, cible déjà dans le piège.                   |
| B22    | M4  | Dégâts, soins, impulsion, brûlure, Mouillé et ralentissement.                  | Mesures, cumul non additif et réactions conformes.                           |
| B23    | M4  | Scène d'épreuve, récepteurs et contrôles de visée.                             | Tous les scénarios reproductibles sans système de quête.                     |
| B24    | M5  | Renderers des six porteurs, quatre matières et géométrie du dessin.            | Aucun rendu générique universel ; superposition collision/rendu.             |
| B25    | M5  | Audio, impacts, extinction et signature persistante du parchemin.              | Rechargement identique et revue humaine de paires de dessins.                |

| Ticket | Lot | Réalisation attendue                                                    | Preuve de sortie                                                 |
|--------|-----|-------------------------------------------------------------------------|------------------------------------------------------------------|
| B26    | M5  | Interface française terminée, focus, clavier et adaptation des tailles. | Parcours complet sans l'éditeur, sur résolutions cibles.         |
| B27    | M6  | Déploiement privé, secrets, sauvegarde, restauration et métriques.      | Restauration à blanc et rapport de panne fournisseur.            |
| B28    | M6  | Performance et endurance sur matériel nommé.                            | Captures profiler, saturation contrôlée et mémoire stable.       |
| B29    | M7  | Corpus final inédit, comparaisons humaines et compilation autonome.     | Preuves de dessin → vrai modèle → sort réel pour chaque famille. |
| B30    | M7  | Livraison, licence des assets, manuel et dossier de recette.            | Installation sur machine propre et validation humaine signée.    |

## Dépendances qui ne doivent pas être contournées

B10 n'attend pas le rendu final, mais n'autorise pas un résultat feint. B13 peut démarrer sur des descriptions manuelles de diagnostic, mais B29 exige de nouvelles images réelles. B18 à B22 peuvent utiliser des fixtures isolées pour tester la physique ; ces tests ne constituent pas la preuve que A comprend un dessin.

B24 et B25 ne peuvent pas se terminer avant que les données géométriques et les événements physiques soient stables. B29 ne peut pas être approuvé par le développeur automatique. B30 n'est pas un simple ZIP de sources : la livraison logicielle finale comprendra le build autonome et la procédure de déploiement décrits au chapitre 23.

**ANNEXE D / SP-1.0**

# Consigne prête pour le développeur

Tu dois réaliser la version finale d'un sous-système de jeu, pas un prototype : dessin de parchemin → vraie interprétation multimodale → description → plan de sort → compilation contrôlée → sort réellement lançable dans Unity. Ne développe pas le reste d'un RPG.

## Lecture préalable obligatoire

Lis `docs/01_CAHIER_DE_REALISATION.md`, `docs/02_ANNEXE_CONTRATS.md`, les schémas de `contracts/`, le catalogue, les prompts A/B et `examples/README.md`. Les exemples sont manuels et ne doivent jamais être présentés comme un résultat de modèle ou de Unity. La QA de ce dossier vérifie des contrats, pas un moteur déjà implémenté.

Unity est choisi. Les brouillons, gomme, annulation, presets de sorts, confirmations après découverte du résultat et rerolls joueur sont exclus. Le modèle lit réellement l'image complète, avec une image de référence. Les pinceaux comptent par leur trace, pas par un rôle tactique codé. Une interprétation peut donner un mauvais sort ; une panne technique ne doit jamais être masquée par un sort de secours.

## Architecture à construire

Client Unity 6.3 LTS / URP Windows x64. Serveur ASP.NET Core .NET 10. Noyau partagé compatible .NET Standard 2.1. Modules A et B séparés et inspectables. Compilateur déterministe. Runtime Unity sans génération de code libre. Stockage durable et reprise de tâches. Six porteurs et tous les effets du catalogue sont inclus dans la livraison finale.

La version exacte d'éditeur, les packages, les bibliothèques de sérialisation et la chaîne de build sont vérifiés puis verrouillés. Ne suppose pas que .NET 10 est le runtime de Unity. Ne charge aucune DLL ou shader venant d'un modèle. Ne livre pas seulement des scripts exigeant de reconstruire les scènes manuellement.

## Méthode de travail

Commence par un audit concret de l'environnement : fichiers disponibles, Unity réellement installé ou absent, version et licence de build, outils .NET, possibilité de construire Windows et accès au fournisseur. Distingue « prévu », « implémenté », « compilé », « testé dans Unity », « testé avec un vrai appel » et « accepté par les créateurs ».

Respecte l'ordre M0 à M7 du cahier. Les tâches techniques indépendantes peuvent avancer, mais ne transforme jamais un jalon humain en auto-validation. À chaque jalon, fournis le build ou outil lançable, les changements, les commandes, les preuves obtenues, les défauts et le test précis que les créateurs doivent effectuer. Ne t'arrête pas à la description textuelle ou à une scène de sphères génériques : l'objectif final inclut géométrie, collisions, effets, rendu, son, sauvegarde et reprise.

Toute fonction exposée au modèle doit être réellement implémentée. Une fonction manquante reste une tâche ouverte ; elle ne devient pas un succès de test en étant remplacée par un mock. Les mocks sont autorisés seulement dans les tests explicitement nommés et ne valident pas le chemin produit.

## Tests et qualité

Réalise tests de contrat, tests unitaires du noyau, tests d'intégration backend, tests de scène, build autonome, appels réels et cas de panne. Garde un corpus de validation inédit distinct des dessins utilisés pour régler les prompts. L'agent ne rédige pas le verdict esthétique ou de gameplay à la place de l'équipe. Des sorties JSON valides ne prouvent pas une bonne lecture du dessin.

La géométrie doit venir des pixels et agir réellement sur au moins une dimension majeure du sort. Une simple couleur ou un nom différent n'est pas une variation suffisante. La description factuelle finale doit dériver des règles compilées. Les signatures sont figées et les sorts déjà créés se lancent sans nouvel appel fournisseur.

## Remise attendue

Un dépôt complet, un client Windows autonome, le backend installable, les assets et scènes, la configuration d'exemple sans secrets, les migrations, les scripts de build/test, la procédure de restauration, les exports de cas et les défauts résiduels. Fournis une preuve de création depuis un dessin inédit, de lancement, de relance hors ligne, et de reprise après panne sans deuxième support ou publication.

Ne déclare pas le produit final « validé » avant les essais humains requis. Si ton environnement ne permet pas un test, indique ce qui n'a pas été exécuté et prépare le test exact ; n'invente ni rapport Unity, ni benchmark, ni appel réel. Continue les travaux réalisables sans prétendre que les preuves manquantes existent.